MDPI

Article

Exact Spanning-Tree Methods for Uzbek Dependency Parsing

Sanatbek Matlatipov

National University of Uzbekistan named after Mirzo Ulugbek, 4 University Street, Tashkent 100174, Uzbekistan; s.matlatipov@nuu.uz

Abstract

Dependency parsing recovers the grammatical structure of a sentence as a rooted spanning tree of its words. We study it for Uzbek, an agglutinative low-resource language, on the spanning arborescences of a complete weighted digraph. For uniformly random dependency trees we derive the covariance of total arc length and crossings and count projective trees by head-final arcs and root position. Against these laws and random reorderings of the observed trees, the 1184 gold Uzbek trees have arcs half as long as expected, about one percent of the expected crossings and a root at the last content word in 73% of sentences. Parsing combines exact decoding by Edmonds' optimum branching algorithm with the directed matrix-tree theorem, which gives the probability that a locally trained parser outputs a well-formed tree. The tree distribution is calibrated by a convex problem solved by a safeguarded Newton method with exact second derivatives. For seven neural parsers on a contamination-free test set, exact decoding takes under 0.2 ms per sentence, calibration lowers the expected calibration error from 16.0% to 7.3% without changing any tree, and their calibrated mixture raises the labeled attachment score on UzUDT from 56.0% to 57.2% (61.6% with gold tags).

Keywords: dependency parsing; spanning arborescence; optimum branching; matrix-tree theorem; random trees; projective trees; Lagrange inversion; probability calibration; Uzbek language

MSC: 05C05; 05A15; 05C80; 05C85; 68T50; 90C27

1. Introduction

Syntactic analysis tells a computer which word of a sentence is the predicate, which words are its arguments and which words modify which. In the dependency representation used by the Universal Dependencies (UD) project [1], this information is a rooted tree whose vertices are the words and whose arcs lead from a governing word, the head, to a dependent word, each arc carrying the name of a grammatical relation. Such trees are used as inputs of many downstream models; recent examples include syntax-aware graph convolutional networks for aspect-based sentiment analysis [2,3], syntax- and morphology-enriched machine translation for low-resource and morphologically rich languages [4,5], and mathematical models of idioms and of semantic linking for Turkic languages [6,7].

From a mathematical point of view, dependency parsing is an optimization problem on graphs. If every ordered pair of words is joined by an arc with a weight, the analysis of a sentence is a maximum-weight spanning arborescence [8], the directed analogue of the minimum spanning tree problem whose algorithms and models continue to be studied in this journal [9–12]. This optimum branching problem is solved exactly by the algorithm

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the author. Submitted to *Mathematics* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

found independently by Chu and Liu [13] and by Edmonds [14], which runs in quadratic time with the data structures of Tarjan [15]; the additional requirement that the root has exactly one child can be met without loss of efficiency [16,17]. The same graph admits a probabilistic treatment: by the matrix-tree theorem of Kirchhoff [18], in the form for directed graphs due to Tutte [19], the sum over all spanning arborescences of the products of their arc weights is a determinant, which yields normalizing constants and arc marginals in cubic time [20–23]. Arc weights are nowadays produced by neural networks; the deep biaffine scorer [24] implemented in the Stanza toolkit [25] is a standard choice.

For Uzbek, an agglutinative Turkic language with rich suffixal morphology, the resources needed by this approach have appeared only recently. The first UD treebank, UT, contains 500 sentences of news and fiction [26], and the UzUDT treebank of 684 sentences from literary and educational texts was annotated manually by six annotators [27–29]. Monolingual contextual encoders [30,31], morphological analyzers [32,33] and text classifiers [34,35] are available, and neural parsers trained on the two treebanks were introduced in [36–38]. Those studies reported scores on development data and on test splits that differed between experiments, did not quantify how far the neural scores are from well-formed trees, and did not compare the observed trees with random ones. The seven parsers analyzed in this paper are the published checkpoints of those studies; here they are re-evaluated on a contamination-free test set, and the mathematical results, the comparisons with null models and the decoding and calibration experiments are new.

Two questions arise when such a parser is applied to a language with little annotated data. The first is descriptive: how far are the observed trees from random ones, and does the answer depend on whether the random reference keeps the shape of the observed tree? Quantitative studies of syntax usually shuffle the words of an observed tree [39–41]; a decoder, in contrast, searches over all trees, so the uniform distribution on the feasible set is an equally natural reference. The second question is numerical: a parser trained with locally normalized scores defines a distribution over head assignments rather than over trees, and it is not obvious how much probability it wastes on ill-formed structures, how its scores should be turned into a distribution over trees and whether the resulting probabilities are reliable.

This paper studies Uzbek dependency parsing as a problem of combinatorics and numerical analysis on spanning arborescences. Its contributions are as follows.

1. Exact laws of two null models for single-root dependency trees: uniformly random trees and random linear arrangements of a fixed tree. Known laws are collected as lemmas with short proofs adapted to the single-root setting. The new results are closed forms for the total arc length and the number of crossings of a uniformly random tree, including their covariance, and a theorem that counts projective trees by the number of head-final arcs and by the position of the root, which gives the triangle A371395 of the On-Line Encyclopedia of Integer Sequences an interpretation in terms of dependency trees. All formulas are verified by exhaustive enumeration (Section 4).
2. A comparison of the 1184 gold Uzbek trees with both null models, which shows that their short arcs, the near absence of crossings and their head-final projective structure are not consequences of tree shape or of projectivity alone (Section 4.3).
3. Numerical methods for the spanning-tree distribution: exact single-root decoding; a probabilistic interpretation of the partition function of a locally normalized model, with an exact relation between arc marginals and local probabilities; two-sided bounds for the determinant and the condition number of the Laplacian matrix along the tempered path; trace formulas for the means and covariances of additive functionals of a random tree; and a convex calibration of the tree distribution that never changes the decoded tree, reduces to the uniform null model at zero inverse temperature and

is solved by a safeguarded Newton method with quadratic convergence (Sections 6 and 7).

4. Computational experiments with seven trained parsers, which differ in the encoder and in the aggregation of subword vectors (Section 5), on a contamination-free test set: significance tests corrected for multiple comparisons, a comparison of decoders and of calibration methods for unlabeled and labeled arcs with bootstrap intervals, mixtures of the calibrated tree distributions of the parsers, which give the best scores of the study without retraining, and a perturbation analysis (Sections 8–10).

2. Related Work

2.1. Spanning Trees, Arborescences and Their Enumeration

The number of spanning trees of a graph is the determinant of a reduced Laplacian matrix by the matrix-tree theorem of Kirchhoff [18], and Tutte [19] proved the corresponding statement for the spanning arborescences of a directed graph, on which the computations of Section 6 rest. In the directed setting, the minimum-cost arborescence problem has polynomial-time algorithms, while constrained variants become NP-hard and call for compact integer models [42]; recent work also studies the efficient answering of many directed minimum spanning tree queries, an application of which is syntactic parsing [9]. Exact enumeration of restricted classes of trees uses Prüfer codes [43], Moon's formula for the number of trees that contain a given forest [44,45], generating functions and Lagrange inversion [46]. Non-crossing trees on points in convex position are counted in [47], the number of projective dependency trees appears as sequence A006013 of the On-Line Encyclopedia of Integer Sequences [48], and mildly non-projective classes of dependency structures are studied in [49]. The Fuss–Catalan numbers that appear below have been studied analytically in [50] and combinatorially through m-ary trees and polygon dissections in [51], and Catalan bounds on the number of derivation trees were recently used to restrict grammar induction [52]. Random labeled trees have height of order $\sqrt{n}$ [53], and simply generated random trees and forests are analyzed in [54].

2.2. Random Baselines for Syntactic Trees

Quantitative studies of syntax compare observed trees with random ones. The most common baseline keeps the tree of a sentence and places its words in a uniformly random order, a random linear arrangement. Under this baseline the expected distance between linked words is $(n+1)/3$ [39], the variance of the total distance is known in closed form [55], and natural languages keep linked words much closer than the baseline predicts [41]. Crossing dependencies are far rarer in natural languages than in random arrangements [40], part of this scarcity may follow from the shortness of dependencies [56], and random projective arrangements, a further baseline, are counted and averaged in linear time in [57]. The direction of dependencies interacts with morphological richness and word order freedom across languages [58]. Uniformly random labeled trees enter these studies mainly as a model of tree shape [40]; we derive further exact laws for single-root dependency trees, in which the arc from the root must also be taken into account, and use both baselines side by side.

2.3. Dependency Parsing for Morphologically Rich and Low-Resource Languages

Graph-based parsers score all arcs and search for the best tree [8,59]; deep biaffine scoring [24,25] remains a strong basis, higher-order information can be added with graph neural networks [60] or graph fusion for small domain-specific samples [61], and globally normalized training with the matrix-tree theorem or with tree-structured conditional random fields is an alternative to local training [20,62]. Projective trees can be found by dynamic

programming over words [63] or headed spans [64], or recast as sequence tagging [65]. Pre-trained encoders [66] have been fine-tuned for Arabic dependency parsing [67], and the Turkish BOUN treebank [68] and a UD treebank of Old English [69] show how much attention the annotation of morphologically rich languages requires. For agglutinative languages, the segmentation of words into subword units matters: tokenizer fertility affects downstream quality [70], morphology-aware segmentation improves Kazakh tokenization [71], and stem–suffix boundaries can be learned across Turkic and Uralic languages [72]. Other Turkic and low-resource languages face the same scarcity of annotated data and have been served by multilingual speech recognition [73], compact language models [74], neural part-of-speech tagging [75] and parameter-efficient adaptation of multilingual models [76].

2.4. Calibration and Uncertainty

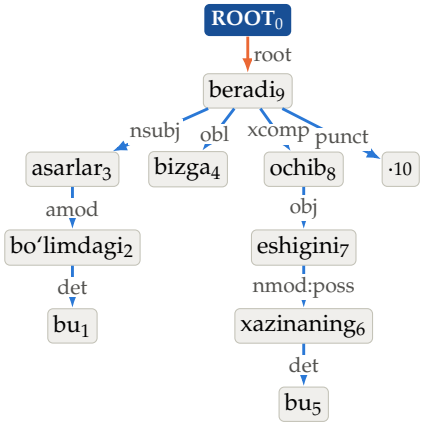
Temperature scaling, a one-parameter rescaling of the logits fitted on held-out data, is a simple and effective remedy for overconfident neural networks [77]; weight averaging offers another route [78], and non-parametric maps such as isotonic regression recalibrate every prediction separately [79]. Calibration is assessed by the expected calibration error, the negative log-likelihood and the Brier score [80], whose limitations are discussed in [81]. For structured outputs, calibration has been studied for sequence labeling, coreference resolution and related tasks of language processing [82–84], mostly by recalibrating the probabilities of individual parts or events. Conditional random fields define globally normalized distributions over structures [85], and the matrix-tree theorem plays this role for dependency trees [20,21]; tempered versions of such distributions, sharpened towards the best structure, were used for annealed minimum-risk training [86]. The log-partition function of every exponential family is convex in its natural parameters [87]; we use this property to calibrate a spanning-tree distribution after training while keeping it a distribution over trees. Finally, the reliability of language processing systems is often probed by controlled input transformations, as in metamorphic testing [88], and differences between systems should be tested with resampling methods [89] corrected for multiple comparisons [90].

3. Mathematical Model

Let a sentence be a sequence of words $x = (w_1, \ldots, w_n)$. Every word is identified with its position $i \in \{1, \ldots, n\}$, and an artificial vertex 0, the root, represents the sentence as a whole. Figure 1 shows a sentence of the UzUDT treebank as it is stored in the corpus and the same record drawn as a tree.

ID	FORM	LEMMA	UPOS	HEAD	DEPREL
1	bu	bu	PRON	2	det
2	bo‘limdagi	bo‘lim	ADJ	3	amod
3	asarlar	asar	NOUN	9	nsubj
4	bizga	biz	PRON	9	obl
5	bu	bu	PRON	6	det
6	xazinaning	xazina	NOUN	7	nmod:poss
7	eshigini	eshik	NOUN	8	obj
8	ochib	och	VERB	9	xcomp
9	beradi	ber	VERB	0	root
10	.	.	PUNCT	9	punct

(**a**) Record in the corpus (CoNLL-U, selected columns)



(**b**) The record as a rooted tree

Figure 1. Sentence s658 of the UzUDT treebank, *bu bo‘limdagi asarlar bizga bu xazinaning eshigini ochib beradi* ‘the works of this section open the door of this treasure to us’. (**a**) The corpus record, in which HEAD is the position of the head of each word and DEPREL the type of the relation; (**b**) the same record as a tree with the relation types on the arcs.

Definition 1 (Sentence graph). *The sentence graph is the complete weighted directed graph*

$$G_x = (V, A, \mathbf{W}), \quad V = \{0, 1, \ldots, n\}, \quad A = \{(j \to i) : i \in \{1, \ldots, n\},\ j \in V \setminus \{i\}\}, \quad (1)$$

where V is the set of vertices, A the set of arcs, $|A| = n^2$, and $\mathbf{W} = (w_{ji})$ the matrix of arc weights, w_{ji} being the weight of the arc from j to i.

Definition 2 (Dependency tree). *A set of arcs $y \subset A$ is a dependency tree of the sentence x if*

$$\deg_y^-(i) = 1 \ \ (i = 1, \ldots, n), \qquad y \text{ contains no cycle}, \qquad \deg_y^+(0) = 1, \qquad (2)$$

where $\deg_y^-(i)$ is the number of arcs of y entering i and $\deg_y^+(0)$ the number of arcs of y leaving the root; $\deg_y^-(0) = 0$ holds automatically, because A contains no arc entering the root.

The first two conditions make y a spanning arborescence rooted at the vertex 0: y has n arcs and no cycle, so every word is reached from the root by exactly one directed path. The third condition is the single-root requirement of UD: exactly one word, usually the main predicate, depends on the root. For the unique arc $(h_i \to i) \in y$ we call h_i the head of i and i a dependent of h_i; a label $r_i \in \mathcal{R}$, the relation type, is attached to every arc ($|\mathcal{R}| = 39$ in our data). A tree is thus encoded by the head vector $h = (h_1, \ldots, h_n)$ and the label vector $r = (r_1, \ldots, r_n)$, and $\mathcal{T}(x)$ denotes the set of all dependency trees of x. An arc $(h_i \to i)$ with $h_i \neq 0$ is head-final if $h_i > i$, that is, if the head follows its dependent. Figure 2 shows the graph of a three-word sentence and one of its trees.

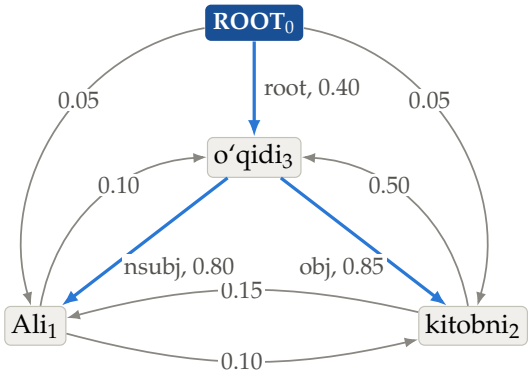


Figure 2. The complete weighted digraph G_x of the sentence *Ali kitobni o‘qidi* ‘Ali read the book’ with the arc probabilities of the numerical example of Section 7: the gold dependency tree (blue) and the six remaining arcs (grey).

Lemma 1. *For every $n \geqslant 1$ the number of dependency trees of a sentence of n words is $|\mathcal{T}(x)| = n^{n-1}$.*

Proof. *First proof.* Deleting the root and forgetting orientations maps every $y \in \mathcal{T}(x)$ to a labeled tree on $\{1, \ldots, n\}$ together with a distinguished vertex, the child of the root. Conversely, a labeled tree and a distinguished vertex determine a unique orientation of the edges away from that vertex, so the map is a bijection. By Cayley's formula [43,91] there are n^{n-2} labeled trees on n vertices, hence $|\mathcal{T}(x)| = n \cdot n^{n-2} = n^{n-1}$.

Second proof. Put all arc weights equal to one in the single-root matrix-tree theorem (Theorem 3 below). The matrix $\hat{\mathbf{L}}$ then coincides, apart from its first row of ones, with the Laplacian matrix of the complete graph K_n. Expanding the determinant along the first row gives the sum of the n cofactors of that row, each of which equals the number n^{n-2} of spanning trees of K_n by Kirchhoff's theorem [18]; hence $|\mathcal{T}(x)| = \det \hat{\mathbf{L}} = n^{n-1}$. □

The feasible set grows faster than exponentially: there are 10^9 trees for $n = 10$ and about 10^{11} for the mean sentence length $\bar{n} = 11.4$ of the Uzbek treebanks.

The arc weights are produced by a probabilistic model with parameters θ. As in [24], the model is normalized locally: for every word it gives a distribution over the possible heads and, for every arc, a distribution over the labels, so that

$$\log P_\theta(y \mid x) = \sum_{i=1}^{n} \log P_\theta(h_i \mid x) + \sum_{i=1}^{n} \log P_\theta(r_i \mid x, h_i), \qquad w_{ji} = \log P_\theta(h_i = j \mid x). \tag{3}$$

Since the label term can be maximized separately for each chosen arc, parsing reduces to a combinatorial problem and an estimation problem.

Problem 1 (Decoding). *Find $\hat{y} = \arg\max_{y \in \mathcal{T}(x)} W(y)$, where $W(y) = \sum_{(j \to i) \in y} w_{ji}$, and give each arc of $\hat{y}$ the label $\hat{r}_i = \arg\max_r P_\theta(r \mid x, \hat{h}_i)$.*

Problem 2 (Estimation). *Given a treebank $\mathcal{D} = \{(x^{(s)}, y^{(s)})\}_{s=1}^{N}$, find*

$$\hat{\theta} = \arg\min_{\theta \in \mathbb{R}^p} \widehat{R}(\theta), \qquad \widehat{R}(\theta) = \frac{1}{N} \sum_{s=1}^{N} \mathcal{L}\big(\theta; x^{(s)}, y^{(s)}\big), \tag{4}$$

where $\mathcal{L}$ is the loss of Section 6.4 and p the number of trainable parameters.

The quality of the analyses of a test set with gold trees is measured by the unlabeled and labeled attachment scores

$$\mathrm{UAS} = \frac{1}{N_\mathrm{w}} \sum_i \mathbf{1}[\hat{h}_i = h_i], \qquad \mathrm{LAS} = \frac{1}{N_\mathrm{w}} \sum_i \mathbf{1}[\hat{h}_i = h_i \ \wedge \ \hat{r}_i = r_i], \tag{5}$$

where the sums run over all N_w words of the test set and relation subtypes (the part after a colon, as in *nmod:poss*) are ignored, as in the standard UD evaluation.

4. Exact Laws of Random Dependency Trees

This section compares the gold trees with two random references. Section 4.1 collects known laws with short proofs adapted to single-root trees, Section 4.2 contains the new results, and Section 4.3 applies both to the Uzbek treebanks.

Definition 3 (Null models)**.** *Let $n \geqslant 2$. Under the uniform model U, the tree y is uniformly distributed on $\mathcal{T}(x)$. Under the random-arrangement model RLA of a fixed tree $t \in \mathcal{T}(x)$, a permutation σ of $\{1, \dots, n\}$ is drawn uniformly, the word at the position i is moved to the position $\sigma(i)$, and y is the tree with the arcs $\sigma(h_i) \to \sigma(i)$, where $\sigma(0) = 0$, so that the root stays to the left of the sentence.*

U randomizes both the shape of the tree and the order of the words. RLA keeps the shape of the observed tree, that is, its numbers of dependents, its height and its depth profile, and randomizes only the order; it is the standard baseline of quantitative syntax [39,40,57]. The two models are related: relabeling the words by a uniformly random permutation maps the uniform distribution on $\mathcal{T}(x)$ onto itself, so U is the mixture of the RLA models of uniformly random trees, and every expectation under U is the U-average of the corresponding RLA expectation. By the bijection in the first proof of Lemma 1, under U the arcs that do not leave the root form a uniformly random labeled tree on $\{1, \dots, n\}$, and the child of the root is an independent uniform word. Finally, U is the member $\beta = 0$ of the tempered family of Section 6.3, which connects the combinatorics of this section with the numerical methods of Section 6.

4.1. Known Laws

Lemma 2. *Let F be a forest on $\{1, \dots, n\}$ with m edges whose components have $c_1, \dots, c_k$ vertices, $k = n - m$. Then exactly $c_1 c_2 \cdots c_k \, n^{k-2}$ labeled trees on $\{1, \dots, n\}$ contain F [44,45], so a uniformly random labeled tree contains F with probability $c_1 \cdots c_k / n^m$. In particular, under U a given pair of distinct words is joined by an arc with probability $2/n$, two given pairs without a common word are both joined with probability $4/n^2$, and two given pairs with one common word with probability $3/n^2$. Every arc $j \to i$ of G_x, the root arcs included, belongs to y with probability $1/n$.*

Proof. The first statement is Moon's formula [44]; a short proof is given in [45]. Dividing by the n^{n-2} labeled trees gives $c_1 \cdots c_k n^{k-n} = c_1 \cdots c_k / n^m$, and the three special cases are forests with the component sizes $(2, 1, \dots, 1)$, $(2, 2, 1, \dots, 1)$ and $(3, 1, \dots, 1)$. The root arc $0 \to i$ belongs to y if and only if i is the child of the root, which has probability $1/n$. The transposition of the labels i and j maps $\mathcal{T}(x)$ onto itself and the arc $j \to i$ onto $i \to j$, so the pair $\{i, j\}$, which is joined with probability $2/n$, is oriented in each direction with probability $1/n$. □

The length of an arc $(h_i \to i)$ with $h_i \neq 0$ is the distance $d = |h_i - i|$ between the positions of the words it connects, $D(y) = \sum_{i: h_i \neq 0} |h_i - i|$ is the total arc length of the tree, and $C(y)$ is its number of crossing pairs of arcs: two arcs with the ends $\{u_1, u_3\}$ and

$\{u_2, u_4\}$ cross if $u_1 < u_2 < u_3 < u_4$, the root arc being drawn from the position 0 at the left of the sentence.

Lemma 3 ([39]). *Under U, and under RLA for every fixed tree, the length d of a uniformly chosen non-root arc of y has the distribution*

$$P(d) = \frac{2(n-d)}{n(n-1)}, \qquad d = 1, \dots, n-1, \tag{6}$$

with mean and variance

$$\mathbb{E}\, d = \frac{n+1}{3}, \qquad \operatorname{Var} d = \frac{(n+1)(n-2)}{18}. \tag{7}$$

Proof. Exactly $n-d$ unordered pairs of words lie at the distance d. Under U, each of them is joined with probability $2/n$ by Lemma 2, and dividing the expected number $2(n-d)/n$ of arcs of length d by the $n-1$ non-root arcs gives (6); under RLA the two ends of an arc occupy a uniformly random pair of positions, which gives the same law. The moments follow from $\sum_{d=1}^{n-1} d^2 = (n-1)n(2n-1)/6$ and $\sum_{d=1}^{n-1} d^3 = (n-1)^2 n^2/4$. □

Lemma 4. *Let $b_n(k) = \binom{n-2}{k} n^{-k}(1-1/n)^{n-2-k}$ be the binomial probabilities with parameters $n-2$ and $1/n$. Under U, the number k of dependents of a uniformly chosen word of y has the distribution*

$$P(k) = \left(1 - \frac{1}{n}\right) b_n(k) + \frac{1}{n}\, b_n(k-1), \qquad k = 0, \dots, n-1, \tag{8}$$

which converges to the Poisson law with unit mean as $n \to \infty$.

Proof. A vertex v occurs $\deg(v) - 1$ times in the Prüfer code of a labeled tree [43], a uniform word of $\{1, \dots, n\}^{n-2}$, so $\deg(v) - 1$ is binomial with parameters $n-2$ and $1/n$. The word v has $\deg(v) - 1$ dependents unless it is the child of the root, which happens independently with probability $1/n$ and adds one dependent. □

Lemma 5. *Under RLA of a fixed tree t with the root word r, let $\deg_t(v)$ be the number of non-root arcs of t at the word v, k_v the number of dependents of v and $A_t = \sum_v \deg_t(v)\big(\deg_t(v) - 1\big)$. Then*

$$\mathbb{E}D = \frac{n^2 - 1}{3}, \qquad \operatorname{Var} D = \frac{(n+1)\big(6(n-1)(n-2) + (n-4)A_t\big)}{180}, \tag{9}$$

$$\mathbb{E}C = \frac{1}{3}\left(\binom{n-1}{2} - \sum_{v=1}^{n} \binom{\deg_t(v)}{2} + n - 1 - \deg_t(r)\right), \tag{10}$$

and t is projective after the rearrangement with probability $\prod_{v=1}^{n}(k_v + 1)!/n!$.

Proof. The mean follows from Lemma 3. The lengths of two arcs of t have, by direct summation over random positions, the covariance $(n+1)(n-8)/180$ if the arcs share a word and $-(n+1)/45$ otherwise; there are A_t ordered pairs of arcs of the first kind and $(n-1)(n-2) - A_t$ of the second, which gives (9), the variance of [55] in our notation. Two non-root arcs without a common word occupy four random positions and cross with probability $1/3$, because exactly one of the three ways to pair four points interleaves; the root arc crosses a non-root arc not incident with r with the same probability, since r must lie between its ends. Counting such pairs gives (10), which without the root term is the expectation used in [40]. In a projective arrangement every subtree occupies an interval, and the interval of v is the concatenation, in any of $(k_v + 1)!$ orders, of v and of the intervals of its dependents; this gives the count of [57]. □

A tree is projective if, for every arc $(h \to i)$, all words between h and i are descendants of h; equivalently, $C(y) = 0$. Projective trees can be decoded by dynamic programming in $O(n^3)$ time [63], so their number measures the size of the search space of such decoders. In a projective tree every subtree covers a contiguous span of words. A span tree, that is, a projective tree covering $k \geqslant 1$ consecutive words, consists of its head word, a possibly empty sequence of span trees covering the words to the left of the head and a possibly empty sequence of span trees covering the words to its right; conversely, every such structure is a projective span tree, and a projective dependency tree is a span tree covering the whole sentence whose head is the child of the root.

Lemma 6. *The number of projective single-root dependency trees on n words is*

$$Q_n = \frac{1}{n}\binom{3n-2}{n-1} = 1,\ 2,\ 7,\ 30,\ 143,\ 728,\ 3876,\ 21318,\ \ldots, \tag{11}$$

the Fuss–Catalan number $\frac{2}{3m+2}\binom{3m+2}{m}$ with $m = n-1$ and the term $a(n-1)$ of the sequence A006013 [48]. Moreover, $Q_{n+1}/Q_n \to 27/4$ and $Q_n/n^{n-1} \to 0$ faster than any exponential.

Proof. By the span-tree decomposition, the generating function $T(z) = \sum_{n\geqslant 1} Q_n z^n$ satisfies

$$T(z) = z\,\phi\big(T(z)\big), \qquad \phi(w) = \frac{1}{(1-w)^2}, \tag{12}$$

and the Lagrange inversion theorem [46] gives $Q_n = \frac{1}{n}[w^{n-1}]\phi(w)^n = \frac{1}{n}[w^{n-1}](1-w)^{-2n} = \frac{1}{n}\binom{3n-2}{n-1}$. The identity $\frac{1}{n}\binom{3n-2}{n-1} = \frac{2}{3n-1}\binom{3n-1}{n-1}$ gives the Fuss–Catalan form [50], and Stirling's formula gives $Q_n = \Theta\big((27/4)^n n^{-3/2}\big)$. □

The OEIS entry [48] records that Q_n counts projective dependency trees, and the decomposition is the classical one for non-crossing structures [47].

Corollary 1. *Under U, a tree is projective with probability Q_n/n^{n-1}, which equals $7/9$ for $n = 3$, 0.0102 for $n = 8$, $1.55 \cdot 10^{-4}$ for $n = 11$ and $9.0 \cdot 10^{-12}$ for $n = 20$.*

Projective decoders thus search a vanishing part of the feasible set: for an eleven-word sentence, about one tree in 6400.

4.2. New Results

Proposition 1. *Under U, the total arc length and the number of crossings have the moments*

$$\mathbb{E}D = \frac{n^2-1}{3}, \qquad \operatorname{Var} D = \frac{(n-2)(n-1)(n+1)(3n-4)}{60n}, \tag{13}$$

$$\mathbb{E}C = \frac{(n-1)^2(n-2)}{6n}, \qquad \operatorname{Cov}(D,C) = \frac{(n-2)(n-1)(n+1)(4n-7)}{180n}, \tag{14}$$

and $(n-1)(n-2)/(3n)$ of the expected crossings involve the root arc. In particular, D and C are positively correlated for $n \geqslant 3$, and the best linear predictor of C given D is $\mathbb{E}C + \kappa_n(D - \mathbb{E}D)$ with the slope

$$\kappa_n = \frac{\operatorname{Cov}(D,C)}{\operatorname{Var} D} = \frac{4n-7}{3(3n-4)} \longrightarrow \frac{4}{9}. \tag{15}$$

Proof. Write $D = \sum_p X_p d_p$ over the $\binom{n}{2}$ pairs p of words, where d_p is the distance of the pair and X_p indicates that it is joined by an arc, and let $s_v = \sum_u |u-v| = \big((v-1)v + (n-v)(n-v+1)\big)/2$. With $S_1 = \sum_p d_p = n(n^2-1)/6$, $S_2 = \sum_p d_p^2 = n^2(n^2-1)/12$ and the sum $A = \sum_v (s_v^2 - \sum_u (u-v)^2) = n(n-2)(n^2-1)(7n+4)/60$ of $d_p d_q$ over the

ordered pairs of distinct pairs with a common word, Lemma 2 gives $\mathbb{E}D = 2S_1/n$ and $\mathbb{E}D^2 = 2S_2/n + 3A/n^2 + 4(S_1^2 - S_2 - A)/n^2$, hence $\operatorname{Var} D = \big(2(n-2)S_2 - A\big)/n^2$, which simplifies to (13). Every set of four positions $u_1 < u_2 < u_3 < u_4$ supports exactly one crossing pair of non-root arcs, $\{u_1, u_3\}$ and $\{u_2, u_4\}$, present with probability $4/n^2$; the root arc $0 \to r$ crosses $\{u_1, u_2\}$ if and only if $u_1 < r < u_2$, and r is uniform and independent of the labeled tree. Hence the non-root arcs contribute $4\binom{n}{4}/n^2 = (n-1)(n-2)(n-3)/(6n)$ crossings on average and the root arc $\sum_r \frac{1}{n} \sum_{u_1<r<u_2} \frac{2}{n} = (n-1)(n-2)/(3n)$. For the covariance, $\mathbb{E}[DC]$ is the sum of $d_p\, P(X_p = X_q = X_{q'} = 1)$ over the pairs p and the crossing pairs (q, q'), plus the sum of $d_p\, P(X_p = X_q = 1)\,(d_q - 1)/n$ over the pairs p and q for the root arc. By Lemma 2, the first probability equals $4/n^2$ if $p \in \{q, q'\}$, $4/n^3$ if p joins q and q', $6/n^3$ if p shares exactly one end with $q \cup q'$ and $8/n^3$ otherwise, and the second equals $2/n$, $3/n^2$ and $4/n^2$ for $p = q$, for pairs with one common word and for disjoint pairs. Summing d_p over the crossing pairs $\{u_1, u_3\}$, $\{u_2, u_4\}$ and the pairs p of each kind gives, with $B_5 = \binom{n+1}{5}$, the sums $4B_5$ for $p \in \{q, q'\}$, $6B_5$ for the four pairs joining q and q', $\frac{20}{3}(n-4)B_5$ for the pairs with exactly one end in $q \cup q'$ and $\frac{5}{6}(n-4)(n-5)B_5$ for the disjoint pairs; the sums of $(d_q - 1)\, d_p$ for the root arc are, with $B_4 = \binom{n+1}{4}$, $2B_4$ for $p = q$, $\frac{2}{5}(7n-16)B_4$ for the pairs with one common word and $\frac{2}{15}(n-3)(5n-11)B_4$ for the disjoint pairs. Each is the sum of a polynomial in the positions over the simplex $u_1 < u_2 < u_3 < u_4$ or $u_1 < u_2$, which the hockey-stick identity $\sum_{u=k}^{m} \binom{u}{k} = \binom{m+1}{k+1}$ reduces to binomial coefficients [92]. Weighting the sums by the probabilities above gives

$$\mathbb{E}[DC] = \frac{(n-3)(n-2)(n-1)^2(n+1)(5n+2)}{90n^2} + \frac{(n-2)(n-1)(n+1)(20n^2-11n-12)}{180n^2},$$

and subtracting $\mathbb{E}D\,\mathbb{E}C$ gives (14); dividing the covariance by $\operatorname{Var} D$ gives (15). All seven sums were also checked symbolically and by the enumeration of Section 4.3. □

The moments of D can also be obtained by averaging Lemma 5 over U, because the RLA mean does not depend on the tree and $\mathbb{E}_{\mathrm{U}} A_t = 3(n-1)(n-2)/n$ by the binomial law of $\deg(v) - 1$; the same averaging of (10) gives $\mathbb{E}C$, whose non-root part is also given in [40]. The root arc contributes the share $2/(n-1)$ of the expected crossings, one fifth for a sentence of eleven words.

The span-tree decomposition can be refined by the direction of the arcs and by the position of the root. Let $N_{n,k}$ be the number of projective trees on n words with exactly k head-final non-root arcs, $N_n^{(r)}$ the number of projective trees whose root word is r, and $t_m = \frac{1}{2m+1}\binom{3m}{m}$, the number of ternary trees with m internal nodes [93].

Theorem 1. *For every $n \geqslant 1$:*

1. $N_{n,k} = \dfrac{1}{n}\dbinom{n+k-1}{k}\dbinom{2n-k-2}{n-1-k}$ *for* $k = 0, \ldots, n-1$;
2. $N_n^{(r)} = t_{r-1}\, t_{n-r}$ *for* $r = 1, \ldots, n$;
3. *exactly* $(n-1)!$ *trees of* $\mathcal{T}(x)$ *have only head-final arcs, and* $\frac{1}{n}\binom{2n-2}{n-1}$ *of them, the Catalan number* C_{n-1}, *are projective.*

Proof. (i) The dependents to the left of a head are joined to it by head-final arcs and the dependents to its right by head-initial arcs. Marking every head-final arc by u, the

generating function $T(z,u)$ of span trees satisfies $T = z\,\phi(T)$ with $\phi(w) = (1-uw)^{-1}(1-w)^{-1}$, so Lagrange inversion gives

$$\sum_k N_{n,k}u^k = \frac{1}{n}[w^{n-1}](1-uw)^{-n}(1-w)^{-n} = \frac{1}{n}\sum_{k=0}^{n-1}\binom{n+k-1}{k}u^k\binom{2n-k-2}{n-1-k}.$$

(ii) No arc of a projective tree crosses the root arc $0 \to r$, so no arc joins a word of $\{1,\dots,r-1\}$ to a word of $\{r+1,\dots,n\}$. Hence a projective tree with the root word r decomposes uniquely into r, a sequence of span trees covering the words $1,\dots,r-1$, whose heads depend on r, and an independent sequence covering $r+1,\dots,n$, and the number of such trees is the product of the numbers of the two sequences. The sequences covering m words have the generating function $H(T) = 1/(1-T)$, and the Lagrange–Bürmann formula gives $[z^m]H(T) = \frac{1}{m}[w^{m-1}]H'(w)\phi(w)^m = \frac{1}{m}[w^{m-1}](1-w)^{-2m-2} = \frac{1}{m}\binom{3m}{m-1} = t_m$ for $m \geqslant 1$, with $t_0 = 1$. (iii) If all arcs are head-final, the word n has no head among the words and is the child of the root, and every word $i < n$ chooses its head among $i+1,\dots,n$; every such choice is acyclic because heads increase along every path, which gives $(n-1)!$ trees, and (i) with $k = n-1$ gives $N_{n,n-1} = C_{n-1}$. □

The numbers $N_{n,k}$ form the row $n-1$ of the triangle A371395, where they are introduced as the graded dimensions of an operad [94]; Theorem 1 shows that they count projective dependency trees by direction, with row sums Q_n and Catalan numbers in the first and last columns.

Corollary 2. *Let y be uniformly distributed on the projective trees on $n \geqslant 2$ words and let K be its number of head-final arcs. Then*

$$\mathbb{E}K = \frac{n-1}{2}, \qquad \operatorname{Var} K = \frac{(n-1)(3n-1)}{4(2n+1)}, \tag{16}$$

which exceeds the variance $(n-1)/4$ of independent equiprobable directions. The root is the last word with probability $n/(3n-2)$ and the first word with the same probability, so it lies at an end of the sentence with probability $2n/(3n-2) \to 2/3$; under U each end has probability $1/n$. All arcs are head-final with probability $\binom{2n-2}{n-1}/\binom{3n-2}{n-1}$.

Proof. With $N_n(u) = \sum_k N_{n,k}u^k = \frac{1}{n}[w^{n-1}](1-uw)^{-n}(1-w)^{-n}$, differentiation under the coefficient gives $N_n'(1) = [w^{n-2}](1-w)^{-2n-1} = \binom{3n-2}{n-2}$ and $N_n''(1) = (n+1)[w^{n-3}](1-w)^{-2n-2} = (n+1)\binom{3n-2}{n-3}$. Dividing by $N_n(1) = Q_n$ gives $\mathbb{E}K = (n-1)/2$ and $\mathbb{E}K(K-1) = (n+1)(n-1)(n-2)/\big(2(2n+1)\big)$, hence (16). By Theorem 1(ii), the root is the last word with probability t_{n-1}/Q_n, and $\binom{3n-2}{n-1} = \frac{3n-2}{2n-1}\binom{3n-3}{n-1}$ gives $t_{n-1}/Q_n = n/(3n-2)$; the first word is symmetric. The last statement is Theorem 1(iii) divided by Q_n. □

Projectivity alone thus makes a root at an end of the sentence far more likely than under U, and it correlates the directions of the arcs.

4.3. Verification and Comparison with Uzbek Trees

All statements about U in Lemmas 3–6, Proposition 1 and Theorem 1 were checked by exhaustive enumeration of the n^{n-1} trees for $n = 1,\dots,8$, which is 2,097,152 trees for $n = 8$, generated from Prüfer codes and choices of the root word: the distributions of arc lengths and dependents, the first two moments of D, the expected numbers of crossings, $\operatorname{Cov}(D,C)$, the numbers Q_n, $N_{n,k}$ and $N_n^{(r)}$ and the numbers of head-final trees agreed exactly, in rational arithmetic, with the formulas. The RLA statements of Lemma 5 were checked on 40 random trees for each $n = 2,\dots,7$ against all $n!$ arrangements of every tree, and the moments (16) against exact sums over Theorem 1(i) for $n \leqslant 39$. Applied to the

sentence lengths of the treebanks, the exact laws also agree with a Monte Carlo sample of 236,800 random trees: the mean arc length is 5.028 (Monte Carlo 5.032), the share of words without dependents 38.5% (38.5%) and the share of projective trees 4.21% (4.17%).

The data are the two UD treebanks of Uzbek. UzUDT (release 2.17) contains 684 sentences with 7582 words from literary and educational texts; it was annotated by six annotators with inter-annotator agreement (Cohen's κ) of 0.91–0.95 for lemmas, part-of-speech tags and morphological features [27,28], and its text is distributed in lower case. UT (release 2.15) contains 500 sentences with 5930 words of news and fiction [26] and keeps the original capitalization. Together they give 1184 trees with 13,512 words. Table 1 and Figures 3–5 compare them with the null models.

Table 1. Graph statistics of the 1184 gold dependency trees and their expected values under the two null models of Definition 3: uniformly random trees with the same sentence lengths (U) and random linear arrangements of the gold trees (RLA).

Characteristic	UzUDT	UT	UzUDT+UT	U	RLA
Sentences	684	500	1184	—	—
Words	7,582	5,930	13,512	—	—
Mean sentence length $\bar{n}$	11.1	11.9	11.4	—	—
Mean arc length	2.60	2.56	2.58	5.03[a]	5.03[a]
Total arc length / expectation	0.49	0.54	0.51	1.00[a]	1.00[a]
Arcs between adjacent words, %[c]	52.4	51.6	52.0	—	—
Head after its dependent, %[c]	88.2	80.4	84.6	50.0[a]	50.0[a]
Crossing pairs of arcs per sentence	0.23	0.14	0.19	19.5[a]	17.2[a]
Non-projective trees, %	11.0	6.8	9.2	95.8[a]	90.3[a]
Mean tree height	—	—	4.33	6.39[b]	4.33[d]
Words without dependents, %	—	—	57.3	38.5[a]	57.3[d]
Words with $\geqslant 4$ dependents, %	—	—	7.8	1.0[a]	7.8[d]

[a] Exact value from Lemmas 3–5, Corollary 1 and Proposition 1, averaged over the 1184 sentences. [b] Monte Carlo estimate from 236,800 random trees. [c] Without punctuation and root arcs (9,897 arcs in UzUDT+UT). [d] The arrangement model keeps every gold tree, hence its height and numbers of dependents.

The mean length of the non-root arcs of the gold trees is 2.58, against 5.03 under both null models, and their total arc length is 0.51 of its expectation; Figure 4a shows that the gap widens with the sentence length. The total arc length is below its expectation in 99.7% of the sentences; in units of the exact standard deviations of Proposition 1 and Lemma 5, it lies on average 2.47 deviations below the mean under U and 2.39 under RLA, and more than two deviations below in 62% and 60% of the sentences, respectively. Among the 9897 arcs that are neither punctuation nor root arcs, 52.0% join adjacent words, and 3.3% are longer than ten words. To describe the shape of this distribution, we fitted laws conditioned on the sentence length: an arc of a sentence of n words has the length d with probability proportional to $(n-d)\,k(d)$, $d = 1, \ldots, n-1$, where $n-d$ is the number of word pairs at the distance d, so that $k \equiv 1$ gives the law of U (Lemma 3) and the kernel k describes the preference for short arcs. Conditioning on n respects the bound $d \leqslant n-1$ and avoids the spurious heavy tail produced by pooling sentences of different lengths [95]. The arcs of one tree are not independent (Proposition 1), so we treat their lengths as marginally distributed by this law and use the composite likelihood of the 9897 arcs, with the Akaike information criterion as a descriptive measure of fit [96]. It equals 40,780 for $k \equiv 1$, 35,119 for the geometric kernel $k(d) = (1-q)^{d-1}$ ($\hat{q} = 0.26$), 34,642 for the Cauchy-type kernel $k(d) = \left(1 + (d-1)^2/s^2\right)^{-1}$ ($\hat{s} = 1.64$) and 32,989 for the power kernel $k(d) = d^{-\gamma}$ ($\hat{\gamma} = 1.18$). Uzbek arc lengths thus follow the uniform law of Lemma 3 tilted by a power of the distance (Figure 3).

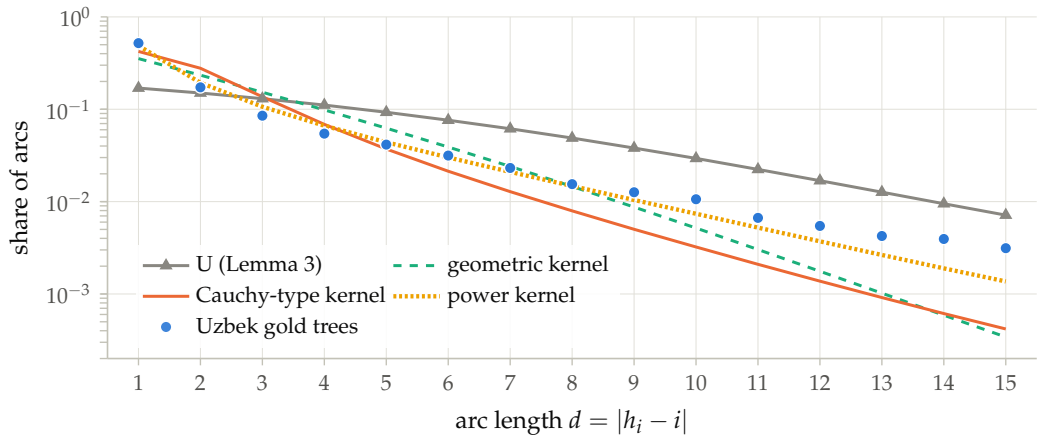


Figure 3. Distribution of the arc length in the Uzbek treebanks (punctuation and root arcs excluded), the laws $(n-d)\,k(d)$ fitted conditionally on the sentence length with geometric, Cauchy-type and power kernels, and the law of U (Lemma 3, $k \equiv 1$), all aggregated over the sentence lengths of the corpus.

The gold trees contain only 222 crossing pairs of arcs, 0.19 per sentence, against 19.5 expected under U and 17.2 under RLA, which is about one percent of the crossings expected by chance even when the shape of every tree is kept (Figure 4c). Short arcs explain part of this deficit. By Proposition 1, crossings and total length are positively correlated in uniformly random trees, and the best linear prediction of C from the observed total length of each gold tree is 8.6 crossing pairs per sentence. To first order, the short arcs thus account for 57% of the missing crossings, but the prediction is still 45 times the observed value. Projectivity separates the distributions most sharply (Figure 4d). The share of projective gold trees decreases slowly with the sentence length, from 95.1% for 6–10 words to 81.8% for 16–20 words and 71.4% for 21–30 words. Random trees of these lengths are projective with probabilities of order 10^{-2}, 10^{-8} and 10^{-13} under U and 10^{-1}, 10^{-7} and 10^{-10} under RLA. Keeping the shape of the trees raises the probability of projectivity by one to three orders of magnitude, but it remains negligible for long sentences. This agrees with the scarcity of crossings observed in other languages [40]. Overall, 9.2% of the gold trees are non-projective (11.0% in UzUDT and 6.8% in UT): projective trees are the rule, but a projective decoder cannot represent about every eleventh sentence.

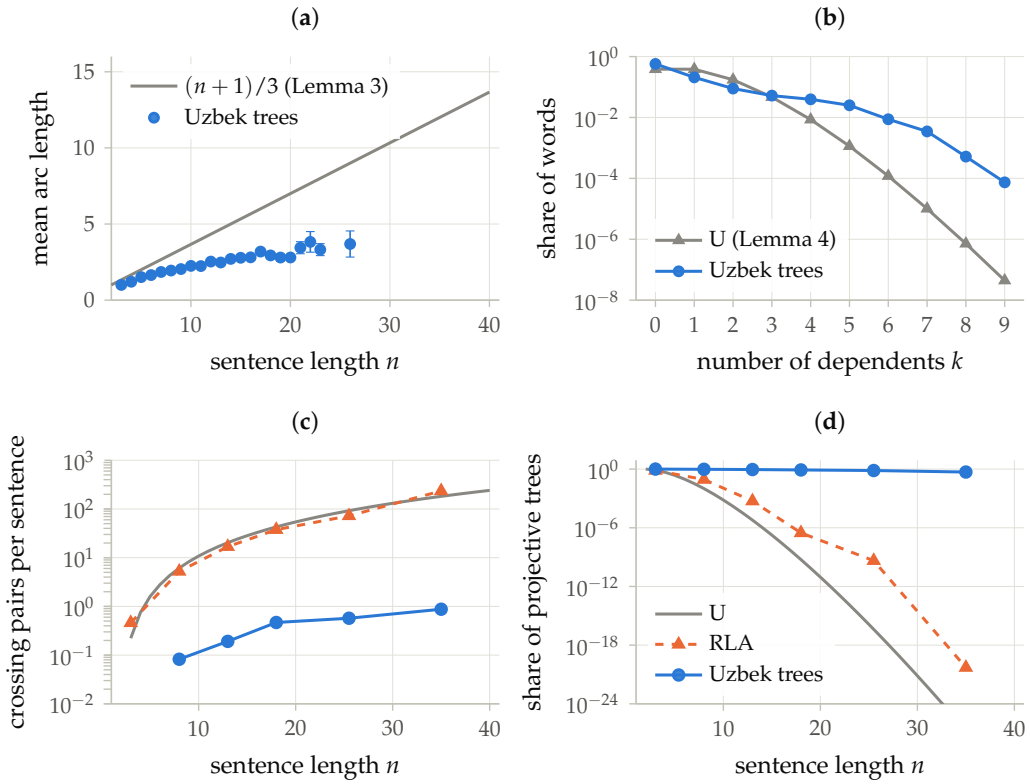


Figure 4. Shape of the gold trees compared with the null models U (uniformly random trees with the same lengths) and RLA (random arrangements of the gold trees). (**a**) Mean arc length against the sentence length with 95% confidence intervals and the expectation of Lemma 3, common to both models; (**b**) distribution of the number of dependents of a word and the exact law of Lemma 4 (RLA keeps the observed distribution); (**c**) mean number of crossing pairs of arcs per sentence in bands of sentence length and the expectations of Proposition 1 (U) and Lemma 5 (RLA); (**d**) share of projective trees in the same bands and the probabilities of Corollary 1 (U) and Lemma 5 (RLA). The legend of (**d**) also applies to (**c**).

Without punctuation, the head follows its dependent in 84.6% of the arcs (88.2% in UzUDT and 80.4% in UT), against one half under both null models by the reflection symmetry $i \mapsto n+1-i$; this is the head-final order of Uzbek, in line with the cross-linguistic interplay of morphology and dependency direction [58]. Theorem 1 measures this order against projective trees. Punctuation tokens are leaves in UD, so deleting them leaves a tree on the content words, and 1075 of these trees are projective. In them, 84.8% of the arcs are head-final; summed over the trees, the number of head-final arcs exceeds its expectation $(n-1)/2$ by 55 standard deviations of Corollary 2, and 38.1% of the trees are entirely head-final, against 5.1% for uniformly random projective trees of the same lengths (Figure 5a). The root is the last content word in 73.1% of the sentences and the first word in 0.5%. By Corollary 2, a uniformly random projective tree of these lengths has its root at the last word with probability 36.8% and at the first word with the same probability, against 13.6% under U (Figure 5b). Uzbek therefore keeps the preference for an end of the sentence that projectivity induces, and it uses only the right end.

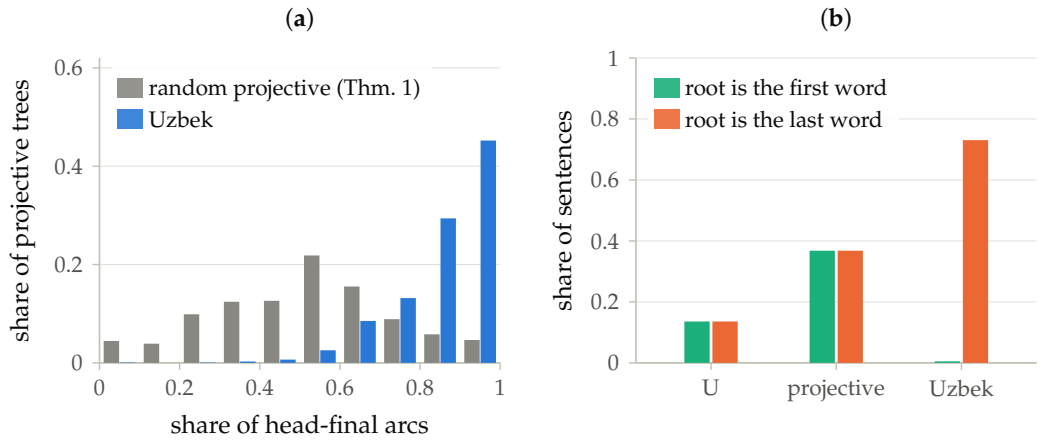


Figure 5. Direction and root position of the projective trees on content words. (**a**) Distribution of the share of head-final arcs in the 1075 projective gold trees and in uniformly random projective trees of the same lengths (exact law of Theorem 1); (**b**) share of sentences whose root is the first or the last word under U, for uniformly random projective trees (Corollary 2) and in the Uzbek treebanks.

The mean height of the gold trees is 4.33, against 6.39 for uniformly random trees, whose height grows like $\sqrt{n}$ [53]. The number of dependents departs from the law of Lemma 4 at both ends (Figure 4b): 57.3% of the words have no dependents against 38.5%, and 7.8% have four or more against 1.0%. Of these 1051 words, 80.1% are verbs, 13.8% nouns and 4.5% adjectives, that is, mostly predicates with their arguments and adjuncts. These findings agree with the design of the arc-weight model of the next section, whose scores contain a direction term and a heavy-tailed distance kernel, and they show that the decoder must admit non-projective trees.

5. Neural Model of Arc Weights

The weights w_{ji} are computed by the graph-based parser of the Stanza toolkit [25], which follows the deep biaffine architecture [24] with the additions of [97]. For Uzbek the parser was extended with a contextual channel: the vectors of a pretrained BERT encoder [66] enter the input layer through an aggregation operator (Figure 6). The extension raises the dimension of the input vector of a word from 500 to 1143 and the number of trainable parameters from 24.5 to 27.6 million; the encoder itself, about 109 million parameters, is kept frozen (Table 2).

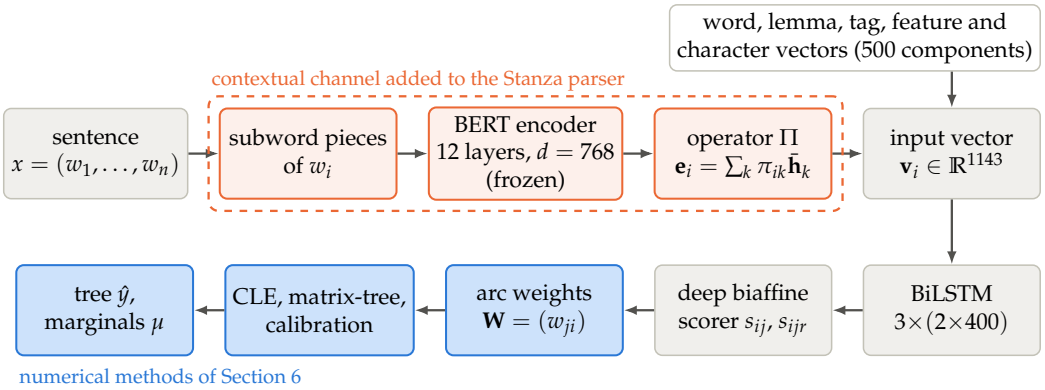


Figure 6. Architecture of the arc-weight model. The orange frame marks the contextual channel added to the Stanza parser; the blue blocks compute the weights $\mathbf{W}$ of the sentence graph and solve the problems of Section 6.

5.1. Subword Vectors and the Aggregation Operator

A BERT encoder splits every word into subword pieces; the word i receives the pieces with the indices $a_i, \dots, b_i$ of the piece sequence of the sentence, $m_i = b_i - a_i + 1$ is the number of its pieces and $M = \sum_i m_i$ the number of pieces of the sentence. As in Stanza, the contextual vector of the piece k is the average of upper hidden layers, $\bar{\mathbf{h}}_k = \frac{1}{4} \sum_{\ell=9}^{11} \mathbf{H}_k^{(\ell)} \in \mathbb{R}^{768}$, where $\mathbf{H}_k^{(\ell)}$ is the output of layer ℓ. The parser, however, needs one vector per word.

Definition 4 (Aggregation operator). *An aggregation operator is a matrix* $\Pi = (\pi_{ik}) \in \mathbb{R}^{n \times M}$ *with* $\pi_{ik} \geqslant 0$, $\sum_k \pi_{ik} = 1$ *and* $\pi_{ik} = 0$ *for* $k \notin [a_i, b_i]$*; the word vectors are* $\mathbf{e}_i = \sum_k \pi_{ik} \bar{\mathbf{h}}_k$*. We consider the last-piece and the mean operators*

$$\pi_{ik}^{\text{last}} = \mathbf{1}[k = b_i], \qquad \pi_{ik}^{\text{mean}} = \frac{\mathbf{1}[a_i \leqslant k \leqslant b_i]}{m_i}. \tag{17}$$

Remark 1. *For each word the admissible rows of* Π *form a simplex whose vertices are the unit vectors with the indices* $a_i, \dots, b_i$*; the row of* Π^{last} *is the vertex* b_i *and the row of* Π^{mean} *the barycenter. By the triangle inequality, every aggregation operator is non-expansive,* $\|\mathbf{e}_i\| \leqslant \max_{a_i \leqslant k \leqslant b_i} \|\bar{\mathbf{h}}_k\|$ *in any norm, and under* Π^{mean} *the weight of the last piece is* $1/m_i$*.*

In Uzbek, the markers of number, possession, case and person are suffixes at the right end of the word, so the last piece tends to carry the inflectional information that syntax needs, and averaging dilutes it by the factor $1/m_i$; this motivated the last-piece operator in [36]. The operator does not discard the stem: the input vector (18) below also contains the embeddings of the word form and the lemma and the output of a character-level network, which represent the stem, and the BERT channel adds the contextual representation of the inflected ending. Table 2 shows how the two encoders segment the 13,512 words of the treebanks; the mean number of pieces per word, the fertility of a tokenizer, is known to affect downstream quality [70]. TahrirchiBERT [30] uses a byte-level BPE vocabulary of 30,528 units and splits 57.4% of the words, into 1.90 pieces on average; BERTbek [31] uses a WordPiece vocabulary of 30,000 units and keeps 68.1% of the words whole (1.55 pieces on average). Among the 5620 inflected words whose form is the lemma followed by suffixes, the last piece contains the whole suffix chain in 90.2% of the cases for TahrirchiBERT and in 79.5% for BERTbek; for example, *kitoblarimizdagi* ‘in our books’ is split into *kitob+larimiz+dagi* and into *kitoblari+mizda+gi*. Because BERTbek splits fewer words, the mean operator keeps a larger weight on the last piece (0.81 against 0.66), which is relevant for the results of Section 9. For comparison, Table 2 also lists the tokenizers of two multilingual encoders that cover Uzbek, mBERT [66] and XLM-R [98], although no parser was trained with them. With vocabularies of 119,547 and 250,002 units shared by about a hundred languages, they split 65.7% and 67.2% of the Uzbek words, into 2.32 and 2.07 pieces on average, and mBERT keeps a whole suffix chain in the last piece for only 57.2% of the inflected words; the monolingual tokenizers are therefore the better match for the last-piece operator.

Table 2. Encoders of the arc-weight model and, for comparison, the tokenizers of two multilingual encoders: dimensions, parameters and subword segmentation of the 13,512 corpus words.

	FastText	**TahrirchiBERT**	**BERTbek**	**mBERT**[b]	**XLM-R**[b]
Input vector $\mathbf{v}_i$	$\mathbb{R}^{500}$	$\mathbb{R}^{1143}$	$\mathbb{R}^{1143}$	—	—
Trainable parameters	24,513,613	27,563,156	27,563,156	—	—
Frozen encoder parameters	—	109,486,848	109,081,344	—	—
Tokenizer	—	byte-level BPE	WordPiece	WordPiece	SentencePiece
Subword vocabulary	—	30,528	30,000	119,547	250,002
Mean pieces per word $\bar{m}$	—	1.90	1.55	2.32	2.07
Words kept whole, %	—	42.6	68.1	34.3	32.8
Words split into $\geqslant 3$ pieces, %	—	26.1	15.1	41.4	27.8
Last piece holds all suffixes, %[a]	—	90.2	79.5	57.2	78.7
Weight of the last piece in Π^{mean}	—	0.66	0.81	0.58	0.61

[a] Over the 5,620 inflected words whose form is the lemma followed by suffixes. [b] Multilingual encoders that cover Uzbek; only their tokenizers are compared, no parser was trained with them.

5.2. Contextual Encoder and Biaffine Scoring

The input vector of the word i is the concatenation

$$\mathbf{v}_i = \left[\mathbf{e}_i^{\text{w}};\ \mathbf{e}_i^{\text{lem}};\ \mathbf{e}_i^{\text{upos}} + \mathbf{e}_i^{\text{xpos}};\ \textstyle\sum_f \mathbf{e}_i^{f};\ \mathbf{P}_c\mathbf{c}_i;\ \mathbf{e}_i\right] \in \mathbb{R}^{1143}, \tag{18}$$

of trainable embeddings of the word form (75 components), the lemma (75), the universal and language-specific part-of-speech tags (50) and the morphological features (50), the output $\mathbf{c}_i$ of a character-level LSTM projected to 125 components, and the aggregated BERT vector $\mathbf{e}_i$ (768 components). In the FastText variant, $\mathbf{e}_i$ is replaced by a projection of a pretrained 300-dimensional vector to 125 components, which gives $\mathbf{v}_i \in \mathbb{R}^{500}$. A three-layer bidirectional LSTM with 400 units per direction maps the sequence $(\mathbf{v}_1, \ldots, \mathbf{v}_n)$ to contextual vectors $\mathbf{r}_i \in \mathbb{R}^{800}$, and two perceptrons give the representations of the word as a dependent and as a head, $\mathbf{a}_i^d = \mathrm{ReLU}(\mathbf{P}_d\mathbf{r}_i + \mathbf{b}_d)$ and $\mathbf{a}_i^h = \mathrm{ReLU}(\mathbf{P}_h\mathbf{r}_i + \mathbf{b}_h)$, both in $\mathbb{R}^{400}$. The score of the arc $j \to i$ is the biaffine form

$$s_{ij} = [\mathbf{a}_i^d; 1]^\top \mathbf{U}\, [\mathbf{a}_j^h; 1], \qquad \mathbf{U} \in \mathbb{R}^{401\times 401}, \tag{19}$$

and label scores s_{ijr} are computed in the same way with one matrix $\mathbf{U}^{(r)}$ per label.

Remark 2. *Writing* $\mathbf{U} = \begin{pmatrix} \mathbf{U}_{11} & \mathbf{u}_{12} \\ \mathbf{u}_{21}^\top & u_{22} \end{pmatrix}$ *with* $\mathbf{U}_{11} \in \mathbb{R}^{400\times 400}$ *gives* $s_{ij} = \mathbf{a}_i^{d\top}\mathbf{U}_{11}\mathbf{a}_j^h + \mathbf{u}_{21}^\top\mathbf{a}_j^h + c_i$ *with* $c_i = \mathbf{a}_i^{d\top}\mathbf{u}_{12} + u_{22}$. *The term* c_i *does not depend on* j *and cancels in the distribution* $P(h_i = j) \propto \exp s_{ij}$, *so the biaffine form consists of a bilinear interaction between dependent and head and of a head prior* $\mathbf{u}_{21}^\top\mathbf{a}_j^h$, *which measures how readily the word* j *becomes a head regardless of the dependent.*

5.3. Direction and Distance

Following [97], the score is corrected by a direction term and a distance kernel,

$$\tilde{s}_{ij} = s_{ij} + \log\sigma\big(\lambda_{ij}\,\mathrm{sgn}(j-i)\big) - \log\Big(1 + \tfrac{1}{2}\big(|j-i| - \delta_{ij}\big)^2\Big), \tag{20}$$

where σ is the logistic function, λ_{ij} a predicted direction score and $\delta_{ij} \geqslant 1$ a predicted arc length, both produced by additional biaffine scorers. The last term is the logarithm of a Cauchy-type kernel with a slow polynomial decay. Both terms were introduced in [97] for all languages; Section 4 supports them for Uzbek a posteriori, since Uzbek arcs are mostly

head-final and their lengths follow a heavy-tailed law. The weights of the sentence graph are the local log-probabilities

$$w_{ji} = \log P_\theta(h_i = j \mid x) = \tilde{s}_{ij} - \log \sum_{k \neq i} \exp \tilde{s}_{ik}, \qquad P_\theta(r_i = r \mid x, h_i = j) = \frac{\exp s_{ijr}}{\sum_{r'} \exp s_{ijr'}}. \tag{21}$$

6. Numerical Methods

6.1. Exact Decoding

The decoding Problem 1 is an optimum branching problem with the additional single-root constraint. It is solved by the algorithm found independently by Chu and Liu [13] and by Edmonds [14], often called the Chu–Liu/Edmonds (CLE) algorithm.

Theorem 2 (Chu and Liu [13]; Edmonds [14])**.** *Let G be a directed graph with real arc weights and a root from which every vertex can be reached. Steps 1–4 of Algorithm 1 return a spanning arborescence of G of maximum total weight.*

Algorithm 1 (Optimum branching with root preselection)**. Require:** weight matrix $\mathbf{W} = (w_{ji})$ of the sentence graph G_x
Ensure: dependency tree $\hat{y} \in \mathcal{T}(x)$

1: for every word i choose the heaviest entering arc, $h_i = \arg\max_{j \neq i} w_{ji}$
2: if the arcs $\{h_i \to i\}$ contain no cycle, they form the optimal arborescence; go to step 5
3: otherwise contract a cycle C into one vertex c with $w'_{j \to c} = \max_{i \in C}(w_{ji} - w_{h_i i})$ and $w'_{c \to k} = \max_{i \in C} w_{ik}$, and solve the contracted problem by steps 1–4
4: expand c: the arc entering c enters the vertex $i^* \in C$ at which its maximum is attained, the arc $h_{i^*} \to i^*$ is removed and the other arcs of C are kept
5: if the root has more than one child, then for every child r of the root solve steps 1–4 with $0 \to r$ as the only arc leaving the root, and keep the heaviest tree
6: **return** $\hat{y}$

The contraction weight $w'_{j \to c}$ is the gain of entering the cycle through $j \to i$ instead of through the cycle arc $h_i \to i$. Step 5, which Stanza and other neural parsers use, tries only the words attached to the root in the unrestricted solution. Stanojević and Cohen [17] proved, with an extension of the augmentation theorem of Gabow and Tarjan to directed graphs, that the root arc of an optimal single-root tree is one of the root arcs of an optimal unrestricted arborescence, so this root preselection is exact. They also showed that lowering the weights of all root arcs by a constant $c > n\,(\max w - \min w)$ makes every optimal arborescence single-rooted, so that one unrestricted run of steps 1–4 solves the constrained problem; [16] attains the same cost with the algorithm of Gabow and Tarjan. We implemented both variants. They returned the same optimum on all 2191 weight matrices of the test sets of Section 9 and on 300 random problems with $n \leqslant 7$ checked by exhaustive search. On the trained models root preselection was faster, 0.075 against 0.70 ms per sentence in the same measurement, because the unrestricted optimum already has a single child of the root in about 97% of the sentences; with lowered root weights every word first chooses a non-root head, so the greedy graph always contains a cycle and more contractions are needed.

Proposition 2 (Complexity)**.** *Exhaustive search that evaluates every tree of $\mathcal{T}(x)$ requires $\Theta(n^n)$ operations. One iteration of steps 1–4 of Algorithm 1 requires $O(n^2)$ operations and at most n iterations are needed, so steps 1–4 cost $O(n^3)$ operations, or $O(n^2)$ with the data structures of [15].*

Step 5 multiplies these bounds by the number of children of the root in the unrestricted solution; with the lowered root weights a single run suffices, so the single-root problem is solved in $O(n^3)$ operations, or $O(n^2)$ with [15], in the worst case.

Proof. By Lemma 1 there are n^{n-1} trees, each evaluated in $\Theta(n)$ operations. The greedy step and the contraction examine each entry of $\mathbf{W}$ once, and a cycle of the functional graph $i \mapsto h_i$ is found by a depth-first search in $O(n)$; every contraction removes at least one vertex. □

For comparison we use greedy head selection, which costs $O(n^2)$ operations but may violate (2), and Eisner's dynamic programming [63], which finds the best projective tree in $O(n^3)$ operations and was adapted to the single-root requirement by choosing the best child of the root at the end.

6.2. Global Normalization

Let $\psi_{ji} = \exp w_{ji} > 0$ be the factor of the arc $j \to i$. For a matrix $\boldsymbol{\phi} = (\phi_{ji})$ of factors on the arcs, let $\hat{\mathbf{L}}[\boldsymbol{\phi}] \in \mathbb{R}^{n \times n}$, with rows indexed by heads and columns by dependents, be the matrix

$$\hat{L}_{ji}[\boldsymbol{\phi}] = \begin{cases} \phi_{0i}, & j = 1, \\ \sum_{k=1,\, k \neq i}^{n} \phi_{ki}, & j = i \neq 1, \\ -\phi_{ji}, & j \neq i,\ j \neq 1. \end{cases} \tag{22}$$

It depends linearly on the factors, and we write $\hat{\mathbf{L}} = \hat{\mathbf{L}}[\boldsymbol{\psi}]$.

Theorem 3 (Single-root matrix-tree theorem [19,20])**.**

$$Z(x) = \sum_{y \in \mathcal{T}(x)} \prod_{(j \to i) \in y} \psi_{ji} = \det \hat{\mathbf{L}}. \tag{23}$$

Proof. Let $\mathbf{L}$ be the matrix given by the second and third lines of (22) in all rows, the in-degree Laplacian matrix of the graph of the words without the root. Its columns sum to zero, so its adjugate has rows proportional to $(1, \dots, 1)$ and all cofactors in a column are equal, $C_{1i} = C_{ii}$; by the directed matrix-tree theorem of Tutte [19], C_{ii} is the sum of the weights of the spanning arborescences of the words directed away from i. The matrix $\hat{\mathbf{L}}$ differs from $\mathbf{L}$ only in its first row, which carries the root factors, so the expansion along that row gives $\det \hat{\mathbf{L}} = \sum_i \psi_{0i} C_{1i} = \sum_i \psi_{0i} C_{ii}$, the sum over the child i of the root of the weights of the trees in which i is the only child of the root. The diagonal of $\hat{\mathbf{L}}$ omits the root factors because these arborescences do not use the root, and any row could replace the first one. □

The globally normalized distribution $P_G(y \mid x) = Z(x)^{-1} \prod_{(j \to i) \in y} \psi_{ji}$ has the arc marginals $\mu_{ji} = P_G\big((j \to i) \in y\big) = \partial \log Z / \partial \log \psi_{ji}$. Since $\partial \log \det \hat{\mathbf{L}} / \partial \hat{L}_{ab} = [\hat{\mathbf{L}}^{-1}]_{ba}$,

$$\mu_{0i} = \psi_{0i}\, [\hat{\mathbf{L}}^{-1}]_{i1}, \qquad \mu_{ji} = \psi_{ji} \Big(\mathbf{1}[i \neq 1]\, [\hat{\mathbf{L}}^{-1}]_{ii} - \mathbf{1}[j \neq 1]\, [\hat{\mathbf{L}}^{-1}]_{ij} \Big), \quad j \geqslant 1, \tag{24}$$

and the entropy of the distribution of trees is

$$H(x) = -\sum_{y \in \mathcal{T}(x)} P_G(y \mid x) \log P_G(y \mid x) = \log Z(x) - \sum_{(j \to i) \in A} \mu_{ji} \log \psi_{ji}. \tag{25}$$

The minimum-Bayes-risk (MBR) tree maximizes the expected number of correct heads, $\hat{y}_{\mathrm{MBR}} = \arg\max_{y \in \mathcal{T}(x)} \sum_{(j \to i) \in y} \mu_{ji}$, and is found by Algorithm 1 with the marginals as weights. The same holds for mixtures: marginals are linear in the distribution, so the uni-

form mixture $\bar{P} = M^{-1} \sum_{m=1}^{M} P^{(m)}$ of the tree distributions of M models has the marginals $\bar{\mu} = M^{-1} \sum_m \mu^{(m)}$, and its MBR tree is the tree of Algorithm 1 with the weights $\bar{\mu}$. With calibrated components (Section 6.3), such a mixture combines models whose scores live on different scales without any further parameter. One LU factorization of $\hat{\mathbf{L}}$ with partial pivoting gives $\log|\det \hat{\mathbf{L}}|$ and $\hat{\mathbf{L}}^{-1}$ in $O(n^3)$ operations, after which (24) and (25) take $O(n^2)$ more.

Lemma 7. *Let $c_1, \dots, c_n > 0$ and $\psi'_{ji} = c_i \psi_{ji}$ for all arcs. Then $Z'(x) = c_1 \cdots c_n \, Z(x)$, and P_G, the marginals and the entropy do not change.*

Proof. Every tree contains exactly one arc entering each word, so the product of the factors of every tree is multiplied by $c_1 \cdots c_n$. For the entropy, $\log Z' - \sum \mu_{ji} \log \psi'_{ji} = \log Z + \sum_i \log c_i - \sum \mu_{ji} \log \psi_{ji} - \sum_i \log c_i \sum_j \mu_{ji}$, and $\sum_j \mu_{ji} = 1$. □

In floating-point arithmetic we take $c_i = 1/\max_j \psi_{ji}$, so that the largest factor entering each word equals one. This is the matrix analogue of the shift used to evaluate a log-sum-exp, and it also justifies the renormalization of the Stanza scores in Section 8.2. The conditioning of the scaled matrix along the tempered factors ψ_{ji}^{β} of Section 6.3 is governed by the loss of the best tree against the greedy choice of heads,

$$\Delta = \sum_{i=1}^{n} \max_{j \neq i} w_{ji} - \max_{y \in \mathcal{T}(x)} W(y) \geqslant 0, \tag{26}$$

which is zero exactly when the greedy heads form a single-root tree and is otherwise the total loss accumulated by the contractions and the root step of Algorithm 1.

Lemma 8. *For $\beta \geqslant 0$ let $\hat{\mathbf{L}}_\beta$ be the matrix (22) of the factors ψ_{ji}^{β} scaled by $c_i = 1/\max_j \psi_{ji}^{\beta}$. Then*

$$e^{-\beta\Delta} \leqslant \det \hat{\mathbf{L}}_\beta \leqslant n^{n-1} e^{-\beta\Delta}, \qquad \frac{e^{\beta\Delta/n}}{n} \leqslant \kappa_2(\hat{\mathbf{L}}_\beta) \leqslant n^{3n/2} e^{\beta\Delta}, \tag{27}$$

where κ_2 is the spectral condition number.

Proof. The scaled factors are $\exp\big(\beta(w_{ji} - \max_k w_{ki})\big) \in (0, 1]$, so by Theorem 3 the determinant is $\sum_y \exp\big(\beta(W(y) - \sum_i \max_k w_{ki})\big)$, a sum of n^{n-1} terms whose largest term is $e^{-\beta\Delta}$. All entries of $\hat{\mathbf{L}}_\beta$ lie in $[-1, n-1]$, and only the $n-1$ diagonal entries of the rows $2, \dots, n$ can exceed one in modulus, so its largest singular value satisfies $\sigma_{\max} \leqslant \|\hat{\mathbf{L}}_\beta\|_F \leqslant (n^3 - 2n^2 + 2n)^{1/2} < n^{3/2}$; every column contains an entry of modulus at least one, coming from the largest factor entering the word, so $\sigma_{\max} \geqslant 1$. The smallest singular value satisfies $\sigma_{\min} \leqslant (\det \hat{\mathbf{L}}_\beta)^{1/n}$ and $\sigma_{\min} \geqslant \det \hat{\mathbf{L}}_\beta / \sigma_{\max}^{n-1}$, which gives the bounds for $\kappa_2 = \sigma_{\max}/\sigma_{\min}$. □

The determinant is therefore bounded away from zero for every β when the greedy heads form a tree, and the conditioning deteriorates exponentially in β otherwise. Since $n^{-1}\kappa_1 \leqslant \kappa_2 \leqslant n\,\kappa_1$ for $n \times n$ matrices [99], the same exponential bounds hold, up to a factor n, for the 1-norm condition number κ_1 that is estimated in practice. On the 2191 test weight matrices, $\Delta = 0$ for 90.5% of the sentences and $\Delta \leqslant 4.2$ nats for the others. The condition number $\kappa_1(\hat{\mathbf{L}}_\beta)$ never exceeded 227 for $\beta \leqslant 0.5$ and 649 for $\beta = 1$, with medians between 10 and 20; at the upper end $\beta = 5$ of the bracket used in Section 6.3 it reached $6.4 \cdot 10^7$, where an LU factorization in double precision still retains about eight significant digits.

Proposition 3. *Let $f = (f_{ji})$ and $g = (g_{ji})$ be real functions on the arcs, $F(y) = \sum_{(j\to i)\in y} f_{ji}$ and $G(y) = \sum_{(j\to i)\in y} g_{ji}$, and let $f \circ \boldsymbol{\psi}$ denote the entrywise product. Under P_G,*

$$\mathbb{E}F = \operatorname{tr}\big(\hat{\mathbf{L}}^{-1}\hat{\mathbf{L}}[f \circ \boldsymbol{\psi}]\big), \qquad \operatorname{Cov}(F,G) = \operatorname{tr}\big(\hat{\mathbf{L}}^{-1}\hat{\mathbf{L}}[f \circ g \circ \boldsymbol{\psi}]\big) - \operatorname{tr}\big(\hat{\mathbf{L}}^{-1}\hat{\mathbf{L}}[f \circ \boldsymbol{\psi}]\,\hat{\mathbf{L}}^{-1}\hat{\mathbf{L}}[g \circ \boldsymbol{\psi}]\big). \tag{28}$$

Given $\hat{\mathbf{L}}^{-1}$, a mean costs $O(n^2)$ operations, and K means with their $K \times K$ covariance matrix cost $O(Kn^3 + K^2n^2)$.

Proof. For real s and t, the factors $\psi_{ji}e^{sf_{ji}+tg_{ji}}$ give the partition function $Z(s,t) = \sum_y \prod_{(j\to i)\in y} \psi_{ji}\, e^{sF(y)+tG(y)}$, so that $\partial_s \log Z = \mathbb{E}F$ and $\partial_t\partial_s \log Z = \operatorname{Cov}(F,G)$ at $s = t = 0$. By Theorem 3, $\log Z(s,t) = \log\det \hat{\mathbf{L}}(s,t)$, and by the linearity of $\hat{\mathbf{L}}[\cdot]$ the derivatives of $\hat{\mathbf{L}}(s,t)$ at the origin are $\partial_s\hat{\mathbf{L}} = \hat{\mathbf{L}}[f \circ \boldsymbol{\psi}]$, $\partial_t\hat{\mathbf{L}} = \hat{\mathbf{L}}[g \circ \boldsymbol{\psi}]$ and $\partial_t\partial_s\hat{\mathbf{L}} = \hat{\mathbf{L}}[f \circ g \circ \boldsymbol{\psi}]$. Jacobi's formula $\partial \log\det \hat{\mathbf{L}} = \operatorname{tr}(\hat{\mathbf{L}}^{-1}\partial\hat{\mathbf{L}})$ and the identity $\partial\hat{\mathbf{L}}^{-1} = -\hat{\mathbf{L}}^{-1}(\partial\hat{\mathbf{L}})\hat{\mathbf{L}}^{-1}$ give (28). A trace $\operatorname{tr}(\hat{\mathbf{L}}^{-1}\mathbf{B})$ is an entrywise sum, $O(n^2)$; each covariance needs the products $\hat{\mathbf{L}}^{-1}\hat{\mathbf{L}}[f \circ \boldsymbol{\psi}]$, $O(n^3)$ each, after which every trace of a product of two matrices is again an entrywise sum. □

The first formula is the identity $\mathbb{E}F = \sum \mu_{ji} f_{ji}$ written without marginals. The covariances of pairs of arc indicators can also be read off $\hat{\mathbf{L}}^{-1}$, as in the determinantal descriptions of random spanning trees and in the expectation semirings of [23], but summing them over all pairs of arcs costs $O(n^4)$ operations; the trace form gives the variance of an additive functional at the cost of one matrix product. The variance of the tree weight, $F = G = W$, is what Newton's method needs in Section 6.3.

Proposition 4. *Let the model be locally normalized, $\sum_{j\neq i} \psi_{ji} = 1$ for every word i, and let $\eta = (\eta_1, \ldots, \eta_n)$ be drawn with independent heads, $P(\eta_i = j) = \psi_{ji}$. Then (i) $Z(x) = P(\eta \in \mathcal{T}(x))$; (ii) the global distribution is the local one conditioned on well-formedness, $P_G(y \mid x) = P(\eta = y \mid \eta \in \mathcal{T}(x))$, and $P_G(y \mid x) \geqslant P_\theta(y \mid x)$ for every tree, with equality for all trees if and only if $Z(x) = 1$; (iii) for every arc,*

$$\mu_{ji} = \psi_{ji}\,\frac{Z(x \mid \eta_i = j)}{Z(x)}, \qquad Z(x \mid \eta_i = j) = P\big(\eta \in \mathcal{T}(x) \mid \eta_i = j\big), \tag{29}$$

so $\mu_{ji} > \psi_{ji}$ exactly when fixing the head j of the word i makes a well-formed output more likely.

Proof. By independence, $P(\eta = y) = \prod_{(j\to i)\in y} \psi_{ji}$ for every head vector y; summing over $y \in \mathcal{T}(x)$ gives (i) by (23), and dividing by $Z(x)$ gives the conditional law in (ii). Moreover, $P_G(y \mid x) = P_\theta(y \mid x)/Z(x)$ with $Z(x) \leqslant 1$. For (iii), Bayes' rule gives $\mu_{ji} = P(\eta_i = j \mid \eta \in \mathcal{T}(x)) = P(\eta_i = j)\,P(\eta \in \mathcal{T}(x) \mid \eta_i = j)/P(\eta \in \mathcal{T}(x))$; the conditional probability is the matrix-tree determinant in which the factors entering i are replaced by the indicator of j. □

Proposition 4 turns the determinant into a diagnostic: $1 - Z(x)$ is the probability mass that the trained model wastes on structures with cycles or with several children of the root, and it equals the probability that sampling the heads independently produces an ill-formed output. It also shows that the global distribution raises the probability of every tree, and part (iii) states which marginals rise: those of the arcs that make a well-formed output more likely, typically the arcs of the decoded tree. Section 9 shows that this makes an overconfident local model slightly more overconfident.

6.3. Temperature Calibration of the Tree Distribution

For an inverse temperature $\beta \geqslant 0$, consider the tempered local and global models

$$p_\beta(h_i = j \mid x) = \frac{\exp(\beta w_{ji})}{\sum_{k \neq i} \exp(\beta w_{ki})}, \qquad P_\beta(y \mid x) = \frac{\exp\big(\beta W(y)\big)}{Z_\beta(x)}, \qquad Z_\beta(x) = \sum_{y \in \mathcal{T}(x)} \exp\big(\beta W(y)\big), \tag{30}$$

where $Z_\beta(x)$ is computed by Theorem 3 with the factors ψ_{ji}^β. For $\beta = 1$, $P_\beta = P_G$; as $\beta \to \infty$, P_β concentrates on the trees of maximum weight; and for $\beta = 0$ it is the uniform model U of Section 4, whose arc marginals all equal $1/n$ by Lemma 2. The parameter thus moves the distribution along a path from the null model of Section 4 to the decoder of Algorithm 1. On a development set $\{(x^{(s)}, y^{(s)})\}$ with sentence lengths n_s we choose β by minimizing the negative log-likelihood of the gold trees,

$$F(\beta) = \sum_s \Big(\log Z_\beta(x^{(s)}) - \beta\, W\big(y^{(s)}\big) \Big). \tag{31}$$

Proposition 5. *The function F is convex and infinitely differentiable on $\mathbb{R}$, with*

$$F'(\beta) = \sum_s \Big(\mathbb{E}_\beta W - W\big(y^{(s)}\big) \Big), \qquad F''(\beta) = \sum_s \mathrm{Var}_\beta\, W \geqslant 0, \tag{32}$$

where $\mathbb{E}_\beta$ and Var_β refer to $P_\beta(\cdot \mid x^{(s)})$ and are given by Proposition 3 with the factors ψ_{ji}^β; in particular,

$$F'(0) = \sum_s \Big(\frac{1}{n_s} \sum_{(j \to i) \in A_s} w_{ji} - W\big(y^{(s)}\big) \Big), \tag{33}$$

where A_s is the arc set of the graph of $x^{(s)}$. If at least one gold tree has a smaller weight than the best tree of its sentence, then $F(\beta) \to \infty$ as $\beta \to \infty$; if, moreover, $F'(0) < 0$, that is, if the gold trees are on average heavier than uniformly random trees, then F attains its minimum on $[0, \infty)$ at a unique point $\hat\beta > 0$. For every $\beta > 0$ the most probable tree under P_β is the tree of Algorithm 1.

Proof. For each sentence, $\log Z_\beta = \log \sum_y \exp(\beta W(y))$ is, up to an additive constant, the cumulant generating function of $W(Y)$ for Y uniform on $\mathcal{T}(x)$, so its first and second derivatives are the mean and the variance of W under P_β; this gives (32) and convexity [87]. At $\beta = 0$ the marginals equal $1/n_s$ by Lemma 2, so $\mathbb{E}_0 W = n_s^{-1} \sum_{(j \to i) \in A_s} w_{ji}$, which gives (33). Every term of (31) is non-negative, because $Z_\beta \geqslant \exp(\beta W(y^{(s)}))$, and as $\beta \to \infty$, $\log Z_\beta = \beta \max_y W(y) + O(1)$, so a sentence whose gold tree is not optimal contributes a term that grows linearly in β. Hence F has a minimizer on $[0, \infty)$, which is positive because $F'(0) < 0$. If W were constant on every $\mathcal{T}(x^{(s)})$, then $F'(0) = 0$; hence W is not constant for some sentence, for which $\mathrm{Var}_\beta W > 0$ at every β because P_β has full support, so F is strictly convex and the minimizer is unique. Finally, $\exp(\beta W(y))$ is increasing in $W(y)$ for $\beta > 0$. □

Algorithm 2 (Safeguarded Newton calibration). **Require:** development weight matrices and gold trees; an initial bracket $[\beta_{\min}, \beta_{\max}]$, a start β_0 and a tolerance τ
Ensure: inverse temperature $\hat\beta$

1: while $F'(\beta_{\min}) \geqslant 0$ set $\beta_{\min} \leftarrow \beta_{\min}/2$; while $F'(\beta_{\max}) \leqslant 0$ set $\beta_{\max} \leftarrow 2\beta_{\max}$; project β_0 onto the bracket
2: **for** $k = 0, 1, 2, \ldots$ **do**
3: compute $F'(\beta_k)$ and $F''(\beta_k)$ by (32) and (28), with one LU factorization per sentence

4: if $F'(\beta_k) < 0$ set $\beta_{\min} \leftarrow \beta_k$, otherwise $\beta_{\max} \leftarrow \beta_k$
5: $\delta \leftarrow F'(\beta_k)/F''(\beta_k)$; if $|\delta| < \tau$, **return** $\hat{\beta}$, the projection of $\beta_k - \delta$ onto $[\beta_{\min}, \beta_{\max}]$
6: $\beta_{k+1} \leftarrow \beta_k - \delta$ if $\beta_k - \delta \in [\beta_{\min}, \beta_{\max}]$, otherwise $\beta_{k+1} \leftarrow (\beta_{\min} + \beta_{\max})/2$
7: **end for**

Step 1 guarantees $F'(\beta_{\min}) < 0 < F'(\beta_{\max})$ under the conditions of Proposition 5, which make F' negative near zero and positive for large β [100]. Because F' is non-decreasing, the bracket then always contains $\hat{\beta}$, so the iterates cannot leave the region where the solution lies; near $\hat{\beta}$, where $F'' > 0$, the Newton steps are accepted and converge quadratically. Bisection on F' needs no second derivative but only halves the bracket at every step, so 50 steps are needed to reduce $[0.05, 5]$ below 10^{-14}. The tempered local model is calibrated in the same way with the per-word negative log-likelihood, whose second derivative is the sum over the words of the variances of the head weights under the tempered local distributions. Calibration changes the probabilities but not the decoded tree.

Temperature scaling is the one-parameter case of a log-linear recalibration. For arc functions $f^{(1)}, \dots, f^{(K)}$ with tree sums $F_k(y)$, the distributions $P_\varphi(y \mid x) \propto \exp \sum_k \varphi_k F_k(y)$ form an exponential family. We use the three functions suggested by Section 4,

$$P_\varphi(y \mid x) \propto \exp\big(\varphi_1 W(y) + \varphi_2 \operatorname{Dir}(y) + \varphi_3 \operatorname{Len}(y)\big), \tag{34}$$

where $\operatorname{Dir}(y)$ is the number of head-final arcs of y and $\operatorname{Len}(y) = \sum \log\big(1 + (d-1)^2\big)$, summed over the non-root arcs, is the logarithm of a Cauchy-type kernel of the arc length.

Proposition 6. *Let* $\mathbf{\Phi}(y) = \big(W(y), \operatorname{Dir}(y), \operatorname{Len}(y)\big)^\top$ *be the statistics of the family (34). Its negative log-likelihood* $J(\varphi) = \sum_s \big(\log Z_\varphi(x^{(s)}) - \varphi^\top \mathbf{\Phi}(y^{(s)})\big)$ *is convex, with the gradient* $\sum_s \big(\mathbb{E}_\varphi \mathbf{\Phi} - \mathbf{\Phi}(y^{(s)})\big)$ *and the Hessian* $\sum_s \operatorname{Cov}_\varphi \mathbf{\Phi}$ *given by Proposition 3. The Hessian is positive definite at every* φ *if and only if no non-zero* $\mathbf{c} \in \mathbb{R}^3$ *makes* $\mathbf{c}^\top \mathbf{\Phi}(y)$ *constant on* $\mathcal{T}(x^{(s)})$ *for every* s*. In that case* J *is strictly convex, and it attains its minimum, at a unique point, if and only if the observed statistic* $\sum_s \mathbf{\Phi}(y^{(s)})$ *lies in the interior of the set* $\mathcal{C} = \sum_s \operatorname{conv}\{\mathbf{\Phi}(y) : y \in \mathcal{T}(x^{(s)})\}$*, the Minkowski sum of the convex hulls of the statistics of the sentences.*

Proof. Convexity, the gradient and the Hessian follow as in Proposition 5, since $\log Z_\varphi$ is the cumulant generating function of $\mathbf{\Phi}$ [87]. For $\mathbf{c} \neq 0$, $\mathbf{c}^\top \operatorname{Cov}_\varphi \mathbf{\Phi}\, \mathbf{c} = \operatorname{Var}_\varphi(\mathbf{c}^\top \mathbf{\Phi})$, which vanishes if and only if $\mathbf{c}^\top \mathbf{\Phi}$ is constant on the support $\mathcal{T}(x^{(s)})$ of P_φ; summing over s gives the criterion. The sentences are independent, so the family of joint distributions is a full exponential family with the sufficient statistic $\sum_s \mathbf{\Phi}(y^{(s)})$, whose convex support is $\mathcal{C}$, and its likelihood has a maximizer if and only if the observed statistic lies in the interior of the convex support [101]; strict convexity makes the maximizer unique. □

We fit φ by Newton's method with a backtracking line search. On every development set the iteration converged to a point where the gradient is below 10^{-7}, which by Proposition 6 is the unique minimizer. At the fitted values, the smallest eigenvalue of the Hessian lies between 38.8 and 66.8 and its condition number between 56 and 234, so the three statistics are far from collinear. Unlike temperature scaling, this family can change the most probable tree. Both calibration problems are restrictions of the global matrix-tree likelihood [20,62] to a few parameters, fitted after local training. Temperature scaling of a classifier [77] is the case of a single word with a softmax over its heads, and tempered tree distributions were used during training in [86]; here the family is fitted after training, and because it remains a distribution over trees, its arc marginals stay mutually consistent, which per-arc maps such as isotonic regression [79] do not guarantee.

6.4. Parameter Estimation

The loss in (4) is the mean negative log-likelihood per word of the factorization (3) plus the losses $\mathcal{L}_{\text{dir}}$ and $\mathcal{L}_{\text{dist}}$ of the direction and distance predictors of (20). When two treebanks are merged, the empirical risk, averaged over words, is the convex combination

$$\widehat{R}(\theta) = \lambda\, \widehat{R}_{\text{UzUDT}}(\theta) + (1-\lambda)\, \widehat{R}_{\text{UT}}(\theta), \qquad \lambda = \frac{5148}{9251} \approx 0.556, \tag{35}$$

where λ is the share of UzUDT among the 9251 training words. The risk is minimized by Adam [102]:

$$\begin{gathered} \mathbf{g}_t = \nabla_\theta \widehat{R}_{B_t}(\theta_{t-1}) \min\bigl(1,\, 1/\|\nabla_\theta \widehat{R}_{B_t}(\theta_{t-1})\|_2\bigr), \\ \mathbf{m}_t = \rho_1 \mathbf{m}_{t-1} + (1-\rho_1)\mathbf{g}_t, \qquad \mathbf{v}_t = \rho_2 \mathbf{v}_{t-1} + (1-\rho_2)\mathbf{g}_t^2, \\ \theta_t = \theta_{t-1} - \alpha\, \frac{\mathbf{m}_t/(1-\rho_1^t)}{\sqrt{\mathbf{v}_t/(1-\rho_2^t)} + \varepsilon}, \end{gathered} \tag{36}$$

where $\widehat{R}_{B_t}$ is the risk on a random mini-batch B_t of at most 5000 words, squares and roots are taken componentwise, $\alpha = 3 \cdot 10^{-3}$, $\rho_1 = 0.9$, $\rho_2 = 0.95$ (denoted β_1 and β_2 in [102]) and $\varepsilon = 10^{-6}$. Dropout with rate 0.5 and word dropout with rate 0.33 regularize the network. Every 100 steps the LAS on the development set is computed and the best parameters are stored; after 3000 steps without improvement the method switches to AMSGrad [103], which replaces $\mathbf{v}_t$ in the denominator by the running maximum $\hat{\mathbf{v}}_t = \max(\hat{\mathbf{v}}_{t-1}, \mathbf{v}_t)$ so that the effective step size cannot grow, and after another 3000 steps without improvement training stops. Because the encoder is frozen, the gradient involves only the 27.6 million parameters of the parser, five times fewer than the 137 million parameters of the whole network.

6.5. Verification of the Implementation

All decoders and the matrix-tree routine were compared with exhaustive enumeration of $\mathcal{T}(x)$ on 880 random problems with $n = 2, \dots, 7$. The number of enumerated trees was n^{n-1} in every case; both versions of CLE, Eisner's algorithm (against the best projective tree) and the MBR decoder returned optimal trees in all problems; the values of $\log Z$, μ and H differed from the enumerated ones by at most $1.3 \cdot 10^{-15}$, $9.1 \cdot 10^{-15}$ and $3.6 \cdot 10^{-15}$; for unit weights $\det \hat{\mathbf{L}} = n^{n-1}$ held exactly for $n = 1, \dots, 12$, which is the second proof of Lemma 1 in numerical form; and $\mu_{ji} = \partial \log Z / \partial w_{ji}$ held to $4.4 \cdot 10^{-10}$ under central finite differences. The trace formulas (28) agreed with enumeration on 60 random problems with $n = 2, \dots, 6$ and the three arc functions of (34) to within $4 \cdot 10^{-14}$ for the means and $5 \cdot 10^{-13}$ for the covariances, and the variance of W agreed with central differences of its mean to within $5 \cdot 10^{-9}$. The determinant bounds (27) held on all test matrices for $\beta \in \{0.05, 0.25, 0.5, 1, 2, 5\}$. Another 1560 random problems confirmed the optimality of CLE in the timing experiment of Section 9.

7. Numerical Example

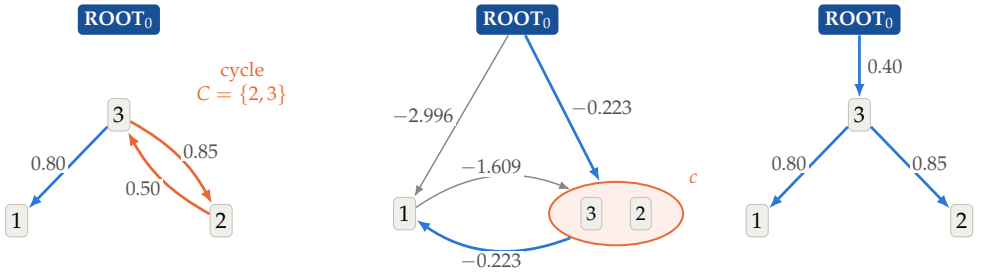
Consider the sentence *Ali kitobni o'qidi* 'Ali read the book' of Figure 2, $n = 3$, and suppose that the model gives the following head distributions, where the row i is the distribution of the head of the word i over $j = 0, 1, 2, 3$:

$$\bigl(P_\theta(h_i = j \mid x)\bigr) = \begin{pmatrix} 0.05 & \cdot & 0.15 & 0.80 \\ 0.05 & 0.10 & \cdot & 0.85 \\ 0.40 & 0.10 & 0.50 & \cdot \end{pmatrix}, \qquad w_{ji} = \log P_\theta(h_i = j \mid x). \tag{37}$$

Decoding. The greedy step of Algorithm 1 chooses $h = (3,3,2)$ (Figure 7a). The arcs $3 \to 2$ and $2 \to 3$ form the cycle $C = \{2,3\}$ and no word is attached to the root, so the greedy result is not a tree. The cycle is contracted into a vertex c (Figure 7b) with the entering weights

$$w'_{0\to c} = \max\Big(\log\frac{0.05}{0.85}, \log\frac{0.40}{0.50}\Big) = \max(-2.833, -0.223) = -0.223,$$
$$w'_{1\to c} = \max\Big(\log\frac{0.10}{0.85}, \log\frac{0.10}{0.50}\Big) = \max(-2.140, -1.609) = -1.609,$$

both attained at the vertex 3, and the arc from c to the word 1 has the weight $\max(w_{21}, w_{31}) = \log 0.80 = -0.223$. In the contracted graph the greedy step chooses $0 \to c$ and $c \to 1$, and there is no cycle. Expanding c, the arc $0 \to c$ enters the vertex 3, so the cycle arc $2 \to 3$ is removed and $3 \to 2$ kept, and $c \to 1$ leaves the vertex 3. The result is $h = (3,3,0)$ (Figure 7c), which coincides with the gold analysis, with the weight $\log(0.80 \cdot 0.85 \cdot 0.40) = \log 0.272$. The loss (26) of this tree against the greedy heads is $\Delta = \log(0.50/0.40) = 0.223$, the contraction weight $w'_{0\to c}$ with the opposite sign. Table 3 lists all $3^2 = 9$ trees of Lemma 1; the tree found is the first one, the maximum.



(**a**) Greedy heads $h = (3,3,2)$ (**b**) Contraction of C into c (**c**) Optimal tree $h = (3,3,0)$

Figure 7. Algorithm 1 on the numerical example. (**a**) Greedy heads with the cycle $C = \{2,3\}$ (orange); (**b**) the contracted graph with the arc weights of the text, the chosen arcs in blue; (**c**) the expanded optimal tree.

Table 3. All $3^2 = 9$ dependency trees of the numerical example: head vectors, arcs, products of arc factors and probabilities under the globally normalized distribution.

Tree	(h_1, h_2, h_3)	Arcs	$\prod \psi$	$P_G(y)$
T_1	$(3,3,0)$	$3\to1, 3\to2, 0\to3$	0.27200	0.7033
T_2	$(2,3,0)$	$2\to1, 3\to2, 0\to3$	0.05100	0.1319
T_3	$(3,1,0)$	$3\to1, 1\to2, 0\to3$	0.03200	0.0827
T_4	$(3,0,2)$	$3\to1, 0\to2, 2\to3$	0.02000	0.0517
T_5	$(0,3,1)$	$0\to1, 3\to2, 1\to3$	0.00425	0.0110
T_6	$(2,0,2)$	$2\to1, 0\to2, 2\to3$	0.00375	0.0097
T_7	$(0,1,2)$	$0\to1, 1\to2, 2\to3$	0.00250	0.0065
T_8	$(2,0,1)$	$2\to1, 0\to2, 1\to3$	0.00075	0.0019
T_9	$(0,1,1)$	$0\to1, 1\to2, 1\to3$	0.00050	0.0013
Sum: $Z = \det \hat{\mathbf{L}}$			0.38675	1.0000

Normalization. With $\psi_{ji} = P_\theta(h_i = j \mid x)$, Theorem 3 gives

$$\hat{\mathbf{L}} = \begin{pmatrix} 0.05 & 0.05 & 0.40 \\ -0.15 & 0.95 & -0.50 \\ -0.80 & -0.85 & 0.60 \end{pmatrix},$$
$$\det \hat{\mathbf{L}} = 0.05 \cdot 0.145 + 0.05 \cdot 0.490 + 0.40 \cdot 0.8875 = 0.38675,$$

which equals the sum Z in Table 3. By Proposition 4, independent sampling of the three heads produces a well-formed tree with probability 0.387 only, and under the global distribution the gold tree has the probability $0.272/0.38675 = 0.703$. Formula (24) gives the marginals $\mu_{03} = 0.918$, $\mu_{31} = 0.838$ and $\mu_{32} = 0.846$; for example, μ_{31} is the total probability of the trees T_1, T_3 and T_4 in which the word 1 depends on the word 3. The MBR tree coincides with the tree found above, and the entropy (25) is $H = 1.02$ nats.

Moments and calibration. At $\beta = 0$ all nine trees of Table 3 are equally likely, and each of the nine arcs lies in exactly three of them, so all marginals equal $1/3$, as Lemma 2 states. At $\beta = 1$, Proposition 3 gives the mean -1.972 and the variance 1.281 of the weight of a random tree, the values of the direct sums over Table 3. The gold tree is the heaviest tree, so $F'(\beta) = \mathbb{E}_\beta W - W(y) < 0$ for every β: calibrating on this sentence alone would drive β to infinity, which is why Proposition 5 requires gold trees that the model does not rank first.

8. Experimental Setup and Software

8.1. Models and Test Sets

Seven trained models were evaluated (Table 4). They are the published checkpoints of [36–38], and their identifiers follow those studies, in which the configuration E4, BERTbek with the last-piece operator, was not trained. The experiments are therefore a post-hoc evaluation of existing checkpoints with the methods of Section 6, not a full factorial design; the comparisons of encoders, operators and training data are restricted to the pairs of models that differ in one of these factors. The models E1.1, E2.1, E3.1 and E5.1 were trained on 435 UzUDT sentences of an earlier split of the treebank with FastText vectors, with TahrirchiBERT and the last-piece operator, with TahrirchiBERT and the mean operator, and with BERTbek and the mean operator, respectively. The models E1.2, E2.2 and E3.2 were trained on the 781 sentences of the merged UzUDT and UT training sets with the first three encoders, with gold tags, and their training files store the Uzbek apostrophes in a different character encoding (UTF-8 bytes read as Windows-1252); the other models were trained with the tags of a separate tagger. Every parser is paired with a tagger trained in the same configuration. Each checkpoint was first applied to the development set of its training run, and the six models with training logs reproduced their best logged development LAS exactly (55.28, 58.23, 55.90, 60.56, 62.87 and 63.81).

The two versions of the UzUDT split used by the models have different test parts. A contamination-free test set was therefore formed from the sentences that belong to the test parts of both versions and have neither exact nor near duplicates (token Jaccard similarity of at least 0.8) in any training or development file: 176 sentences with 1843 words remained after one near duplicate had been removed. The UT test set of 137 sentences and 1421 words has no such overlaps; it belongs to the training genre of the merged models and to a new genre for the other models. Two conditions are reported: tags predicted by the paired tagger, which is an end-to-end analysis of tokenized text, and gold tags. All models read clean text; Section 9 also reports the merged models on input in the encoding of their training files. Differences between models are tested by the paired bootstrap over sentences [89] with $B = 10^4$ resamples; one-sided p-values are adjusted by Holm's method [90] over 16 tests, and a pooled estimate is computed for each factor by stratified resampling.

8.2. Probabilities and Calibration Data

At inference time, Stanza applies the head softmax over the padded length of a batch, so the stored log-probabilities of a word sum over its real heads to slightly less than one (0.95 on average). Because training masks the padding, we renormalize every row over the n real candidate heads before computing local probabilities and $Z(x)$; by Lemma 7, decoding,

marginals and entropies are invariant under this scaling. The inverse temperatures of Section 6.3 were fitted by Algorithm 2 with the bracket $[0.05, 5]$, the start $\beta_0 = 1$ and the tolerance $\tau = 10^{-10}$ on the development set of each model (48 sentences with 644 words for the first four models, 78 sentences with 851 words for the merged models, in clean text), with tags predicted by the paired tagger, and then applied to the test sets; their uncertainty was estimated from 1000 bootstrap resamples of the development sentences. Labels are calibrated in the same way with a separate inverse temperature for the label distributions (21), fitted on the gold arcs of the development sets. The expected calibration error (ECE) is the weighted mean, over ten equal-width probability bins, of the absolute difference between the mean predicted probability and the share of correct predictions; it is computed on the arcs of the decoded trees and, to check the influence of the binning, also with ten equal-mass bins. We also report the Brier score [80], both on the decoded arcs and over all candidate heads of every word, and the negative log-likelihood of the gold trees; intervals for the test values come from 2000 bootstrap resamples of the test sentences. For comparison, the three-parameter family (34) and isotonic regression of the marginals of the decoded arcs, computed by the pool-adjacent-violators algorithm [79], were fitted on the same development data.

8.3. Software Complex

The methods are implemented as a software complex in Python (Figure 8). The training part is based on the tagger and parser of Stanza [25]: the parser received the contextual channel of Figure 6 with a zero vector for the root, the routine that extracts BERT vectors was extended with both operators (17), the data loader and trainer pass the raw word sequence to the encoder, the tagger received the same channel, and the training scripts record the loss and the development scores; in total about 570 lines in 20 source files were changed. A separate spaCy module provides Uzbek tokenization rules and a transition-based pipeline compared with the graph-based parser in [37]. The decoding part contains the decoders of Section 6, the matrix-tree routine, the moment formulas and the calibration methods, together with unit tests against exhaustive enumeration, and the analysis part contains the exact baselines of Section 4, the bootstrap, the perturbation operators and the timing experiments.

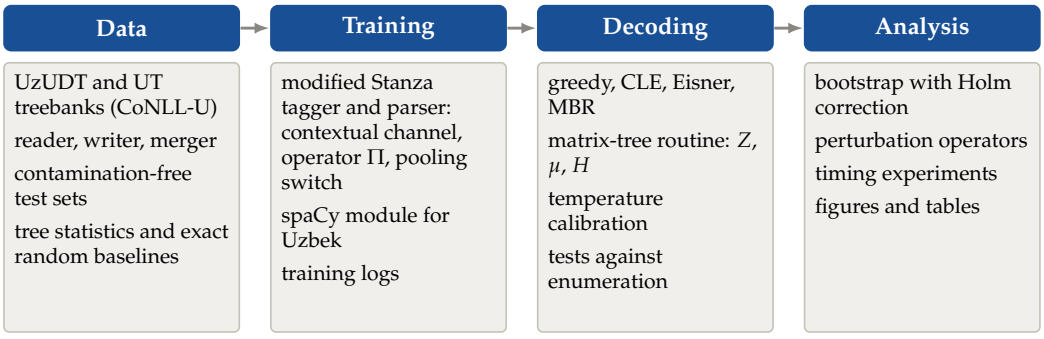


Figure 8. Structure of the software complex.

Training used Python 3.11 and PyTorch 2.6 (CUDA 12.4) on an NVIDIA RTX A6000 graphics processor; inference, calibration and timing used Python 3.12, PyTorch 2.14 and NumPy 2.5 on an Apple M2 Max processor without a graphics card. The encoder libraries must be pinned to the versions used in training, transformers 5.2.0 and tokenizers 0.22.2: with the later release 5.20.2 the development LAS of the BERT-based parsers differed from the training logs by up to 1.6 points, and with the pinned versions every checkpoint reproduced its logged score exactly.

9. Results

9.1. Accuracy and Statistical Significance

The best end-to-end model is E2.2 (TahrirchiBERT, last-piece operator, merged data), with LAS 56.00 on UzUDT and 53.84 on UT, and 60.44 and 59.32 with gold tags (Table 4); among the models trained on UzUDT alone the best is E5.1 with BERTbek (54.64 and 47.71). Figure 9 shows the 95% bootstrap intervals, about ±3 points for a single model, and Table 5 the paired comparisons.

Table 4. Accuracy on the contamination-free test sets with exact decoding, %. Tags are predicted by the paired tagger (pred.) or taken from the gold annotation (gold). The last two rows are the mixtures of calibrated tree distributions of Section 9.4.

			UzUDT Test (176 Sentences)				UT Test (137 Sentences)			
Model	**Vectors**	**Training Data**	**UPOS** pred.	**UAS** pred.	**LAS** pred.	**LAS** gold	**UPOS** pred.	**UAS** pred.	**LAS** pred.	**LAS** gold
E1.1	FastText	UzUDT	83.99	67.39	49.38	54.91	77.90	59.32	41.17	43.49
E2.1	TahrirchiBERT, last	UzUDT	86.81	71.03	53.07	58.11	82.27	60.52	43.98	46.73
E3.1	TahrirchiBERT, mean	UzUDT	85.62	71.68	53.01	57.84	80.72	59.39	42.15	46.09
E5.1	BERTbek, mean	UzUDT	85.35	73.03	54.64	60.28	82.34	63.55	47.71	51.94
E1.2	FastText	UzUDT+UT	85.40	69.02	51.38	57.51	85.22	63.76	50.60	57.28
E2.2	TahrirchiBERT, last	UzUDT+UT	87.30	73.09	56.00	60.44	86.91	67.91	53.84	59.32
E3.2	TahrirchiBERT, mean	UzUDT+UT	86.65	73.03	54.86	60.07	86.07	65.94	51.23	60.10
Mixture of 7	all	both	—	74.23	57.19	61.64	—	66.57	51.65	55.67
Mixture of 3	FastText, TahrirchiBERT	UzUDT+UT	—	73.74	55.40	60.88	—	67.21	53.62	60.66

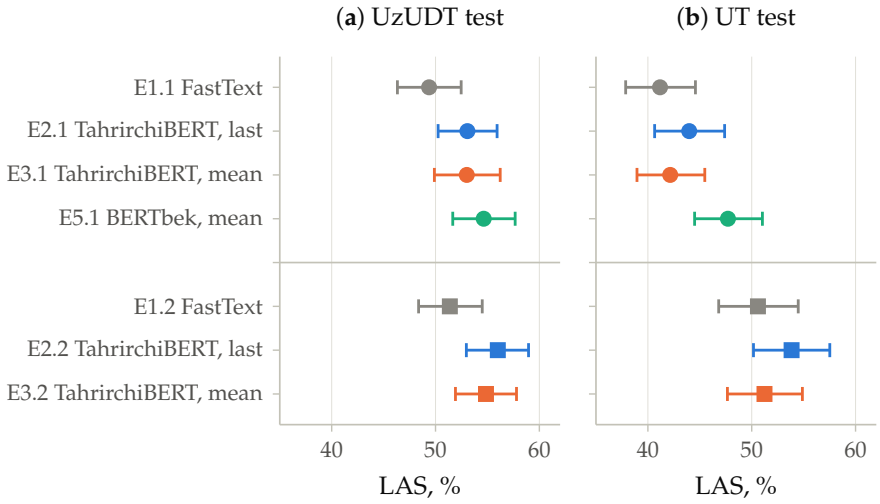


Figure 9. LAS of the seven models on the two test sets with 95% percentile bootstrap intervals over sentences (predicted tags, exact decoding). Colors mark the encoder (grey: FastText; blue: TahrirchiBERT with the last-piece operator; orange: TahrirchiBERT with the mean operator; green: BERTbek with the mean operator); circles mark models trained on UzUDT and squares models trained on UzUDT+UT.

The contextual encoders raise LAS over FastText by 3.69 and 4.61 points on UzUDT, and BERTbek exceeds TahrirchiBERT with the same mean operator by 5.56 points on UT; these differences remain significant after the adjustment, and the pooled encoder effect is +3.55 points with the 95% interval [2.68; 4.43]. The last-piece operator is better than the mean operator in all four comparisons (+0.05 to +2.60 points), none of which is significant on its own after the adjustment; the pooled effect, +1.30 points with the interval [0.33; 2.26] and $p = 0.005$, indicates a small advantage, in agreement with the suffix analysis of Section 5, and it is several times smaller than the encoder and data effects. Training on the merged treebanks raises LAS on UT by 9.1–9.9 points for all three encoders and on UzUDT by 1.8–2.9 points, with a pooled effect of +5.39 points [4.27; 6.55]; this effect combines more data with a different source of training tags, since the merged parsers were trained on gold tags. The comparison with gold tags at test time, where all models receive the same input, isolates the effect of the data: merging raises LAS by 2.23–2.60 points on UzUDT and by 12.59–14.01 points on UT (Table 4). When the merged models read input in the encoding of their training files, the merged-training effect is +3.2 points on UzUDT and +8.9 on UT on average over the encoders, against +2.3 and +9.5 with clean input, so the conclusions do not depend on the encoding. The change is concentrated in the FastText model E1.2 (+2.2 points on UzUDT and −1.6 on UT), whose word-level input depends on the exact form of the words with apostrophes; the TahrirchiBERT models E2.2 and E3.2 change by at most 0.6 points, in line with their insensitivity to missing apostrophes in Section 9.6. For orientation, the best systems of the CoNLL 2018 shared task, which parsed raw text and are therefore not directly comparable, reached LAS 66.44 for Turkish and 67.05 for Uyghur, with treebanks several times larger than UzUDT, and at most 37.01 for treebanks with a few hundred or fewer training sentences such as Armenian and Kazakh [104]. Predicted tags cost 4.4–6.1 LAS points on UzUDT and 2.3–8.9 points on UT compared with gold tags, so tagging errors propagate into the trees. The development scores reported earlier for these models are higher than the present test scores (63.81 against 56.00 for E2.2), mainly because the merged development set was parsed with gold tags; the present values are the appropriate estimates of real-world performance.

Table 5. Paired bootstrap comparisons of LAS (predicted tags, $B = 10^4$ resamples of sentences). The p-values are one-sided; p_{Holm} is adjusted over all 16 tests; † marks differences that remain significant after the adjustment. The factor UzUDT+UT vs. UzUDT also changes the training tags from predicted to gold (Section 8).

Test Set	Factor	Models	ΔLAS	95% Interval	p	p_{Holm}
UzUDT	last vs. mean	E2.1–E3.1	+0.05	[−1.70; +1.86]	0.4930	0.4930
UzUDT	last vs. mean	E2.2–E3.2	+1.14	[−0.52; +2.79]	0.0921	0.2760
UzUDT	encoder	E5.1–E3.1	+1.63	[−0.22; +3.48]	0.0432	0.2340
UzUDT	encoder	E2.1–E1.1	+3.69†	[+1.53; +5.96]	0.0005	0.0055
UzUDT	encoder	E2.2–E1.2	+4.61†	[+2.74; +6.53]	0.0001	0.0016
UzUDT	UzUDT+UT vs. UzUDT	E1.2–E1.1	+2.01	[−0.57; +4.58]	0.0690	0.2760
UzUDT	UzUDT+UT vs. UzUDT	E2.2–E2.1	+2.93†	[+0.77; +5.15]	0.0034	0.0340
UzUDT	UzUDT+UT vs. UzUDT	E3.2–E3.1	+1.84	[−0.59; +4.21]	0.0692	0.2760
UT	last vs. mean	E2.1–E3.1	+1.83	[−0.21; +3.86]	0.0390	0.2340
UT	last vs. mean	E2.2–E3.2	+2.60	[+0.07; +5.18]	0.0246	0.1722
UT	encoder	E5.1–E3.1	+5.56†	[+3.28; +7.82]	0.0001	0.0016
UT	encoder	E2.1–E1.1	+2.81	[+0.37; +5.29]	0.0123	0.0984
UT	encoder	E2.2–E1.2	+3.24	[+0.54; +5.96]	0.0095	0.0855
UT	UzUDT+UT vs. UzUDT	E1.2–E1.1	+9.43†	[+5.89; +12.95]	0.0001	0.0016
UT	UzUDT+UT vs. UzUDT	E2.2–E2.1	+9.85†	[+6.43; +13.38]	0.0001	0.0016
UT	UzUDT+UT vs. UzUDT	E3.2–E3.1	+9.08†	[+6.17; +11.93]	0.0001	0.0016
pooled	last vs. mean	4 strata	+1.30	[+0.33; +2.26]	0.0053	—
pooled	encoder	6 strata	+3.55	[+2.68; +4.43]	0.0001	—
pooled	UzUDT+UT vs. UzUDT	6 strata	+5.39	[+4.27; +6.55]	0.0001	—

9.2. Exact versus Approximate Decoding

Table 6 compares the decoders on the same weight matrices. Greedy head selection leaves 8.6% of the UzUDT sentences and 10.7% of the UT sentences without a valid tree on average, and up to 20.4% for E2.2 on UT; CLE without the single-root step still produces several children of the root in 2.2% and 4.2% of the sentences. The single-root CLE always returns a valid tree with the accuracy of greedy selection (within 0.1 points) in 0.065 ms per sentence on average, so the whole test set of 313 sentences is decoded in about 20 ms. Eisner's algorithm is 3.8 times slower and represents only projective trees; its slightly higher mean LAS reflects the fact that most predicted trees are projective, but on the 27 gold non-projective test sentences CLE is more accurate (44.63 against 44.20 on UzUDT and 44.09 against 43.72 on UT). The MBR decoder gives the accuracy of CLE within 0.1 points. Decoding the MBR tree from the calibrated marginals changes few heads and does not improve the accuracy (LAS 53.09 on UzUDT and 47.05 on UT, against 53.14 and 47.28 at $\beta = 1$); with the marginals of the three-parameter family (34) LAS falls to 52.05 and 46.47.

Table 6. Decoders applied to the same arc-weight matrices of the seven trained models (predicted tags, means over the models). "Valid" is the share of sentences whose output is a single-root tree.

	UzUDT Test		UT Test		Time,
Decoder	**LAS**	**Valid, %**	**LAS**	**Valid, %**	**ms/Sentence**
Greedy head selection	53.21	91.4	47.20	89.3	0.003
CLE, root not restricted	53.22	97.8	47.30	95.8	0.057
CLE, single root (Algorithm 1)	53.19	100.0	47.24	100.0	0.065
Eisner (projective)	53.37	100.0	47.50	100.0	0.248
MBR (CLE on μ)	53.14	100.0	47.28	100.0	0.120

On the 27 gold non-projective test sentences the mean LAS is 44.63/44.09 (UzUDT/UT) for CLE and 44.20/43.72 for Eisner.

Figure 10 shows the time per sentence against the sentence length. Exhaustive enumeration needed 1.5 ms for $n = 5$ and 8.9 s for $n = 8$; extrapolated with the measured cost of 0.52 μs per head assignment, it would need about 1.7 days for $n = 11$, 7200 years for $n = 15$ and $3.2 \cdot 10^{10}$ years for $n = 19$, longer than the age of the Universe. CLE needs at most 0.18 ms per sentence on the matrices of the trained models and about 2 ms for random matrices with $n = 40$, which contain more cycles (on average 0.9 for $n = 11$ and 2.2 for $n = 100$ in the greedy step). In this range of lengths the measured times of the Python implementations are dominated by a constant overhead per call, so they do not reveal the asymptotic orders, which are those of Proposition 2 and $O(n^3)$ for Eisner's algorithm. The reduction of the search from n^{n-1} candidates to a polynomial number of operations thus turns an infeasible problem into one solved in real time.

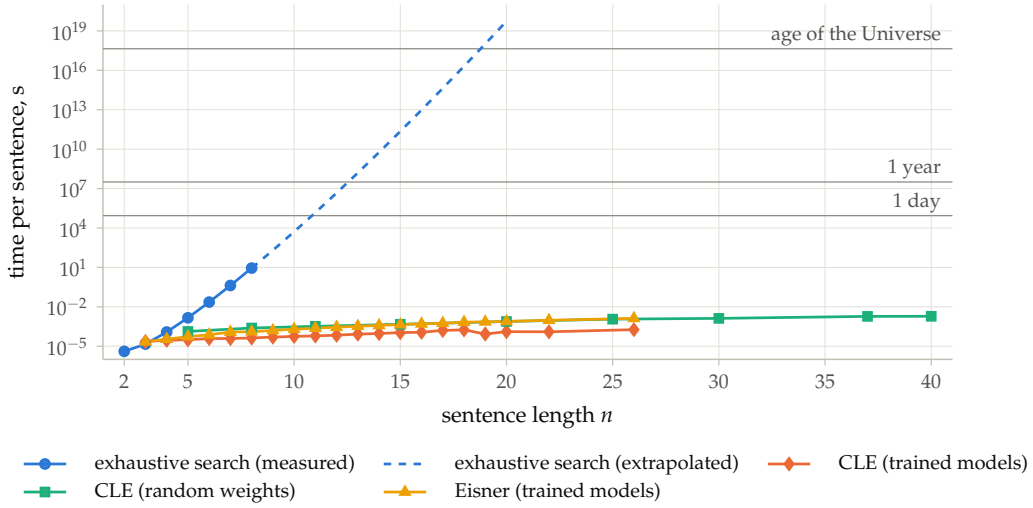


Figure 10. Time needed to find the best tree of one sentence: exhaustive search (measured up to $n = 8$ and extrapolated), CLE on the matrices of the trained models and on random weight matrices, and Eisner's projective algorithm on the matrices of the trained models.

9.3. Matrix-Tree Diagnostics and Calibration

The mean value of $Z(x)$ is 0.84 on UzUDT and 0.81 on UT (medians 0.93 and 0.88): by Proposition 4, the locally trained models spend 16–19% of their probability mass on ill-formed structures, which explains the failures of greedy selection. The models differ in this respect (Table 7): averaged over the two test sets, the value is lowest for the FastText model E1.1 (0.77) and highest for the BERTbek model E5.1 (0.91 and 0.92).

Table 7. Matrix-tree diagnostics and temperature calibration. $\hat{\beta}_G$ is the inverse temperature of the tree distribution fitted on the development set by Algorithm 2, with a 95% bootstrap interval over the development sentences; $\bar{Z}$ is the mean probability of a well-formed tree (UzUDT/UT); ECE, % (10 bins) and the per-word negative log-likelihood of the gold trees are given before→after calibration on the test sets.

			ECE, Local		ECE, Global		NLL, Global	
Model	$\hat{\beta}_G$	$\bar{Z}$	**UzUDT**	**UT**	**UzUDT**	**UT**	**UzUDT**	**UT**
E1.1	0.44 [0.37; 0.52]	0.77/0.77	13.6→5.5	19.2→9.1	15.1→5.4	20.9→10.2	1.40→1.02	1.73→1.19
E2.1	0.46 [0.38; 0.56]	0.84/0.81	11.6→5.9	21.3→8.0	12.7→6.6	22.5→10.0	1.34→0.98	1.83→1.21
E3.1	0.43 [0.36; 0.53]	0.87/0.83	11.0→8.2	20.7→7.6	11.9→8.5	21.4→10.4	1.32→0.96	1.83→1.21
E5.1	0.27 [0.21; 0.32]	0.91/0.92	17.3→6.6	26.7→7.1	17.8→7.5	27.3→8.2	1.69→0.88	2.72→1.15
E1.2	0.40 [0.33; 0.49]	0.83/0.79	11.0→8.5	16.5→6.6	12.0→9.3	18.2→8.0	1.19→0.95	1.46→1.02
E2.2	0.43 [0.36; 0.51]	0.83/0.75	9.4→8.5	14.3→8.1	10.3→8.9	14.9→9.3	1.07→0.87	1.18→0.90
E3.2	0.44 [0.37; 0.54]	0.84/0.79	9.7→6.8	15.9→4.2	10.6→7.8	17.1→5.7	1.06→0.85	1.20→0.91
Pooled ECE (all models, 22,848 arcs)			15.0→6.4		16.0→7.3			

Both the local probabilities and the global marginals overestimate the accuracy (Figure 12); for E2.2 on UzUDT, for example, the mean local probability of a decoded arc is 0.82 and the mean marginal 0.83, while 73.1% of the arcs are correct. Pooled over all models, the ECE is 15.0% for the local probabilities and 16.0% for the global marginals. Conditioning on well-formedness raises the marginal above the local probability for 78.6% of the decoded arcs, so, in agreement with Proposition 4(iii), global normalization makes an overconfident local model slightly more overconfident. The fitted inverse temperatures of the tree distribution lie between 0.27 and 0.46, and their bootstrap intervals, which lie between 0.21 and 0.56, exclude one for every model (Table 7): the models are too sharp; the local inverse temperatures are similar, 0.28–0.49. Started from $\beta_0 = 1$, Algorithm 2 reached the solution of bisection to within $2 \cdot 10^{-15}$ after six or seven evaluations of F' and F'', with the errors $5.6 \cdot 10^{-1}$, $8.5 \cdot 10^{-2}$, $1.9 \cdot 10^{-2}$, $8.7 \cdot 10^{-4}$, $1.8 \cdot 10^{-6}$ and $8.2 \cdot 10^{-12}$ for E1.1, and in 0.02–0.03 s per model against 0.3 s for the 50 steps of bisection (Figure 11).

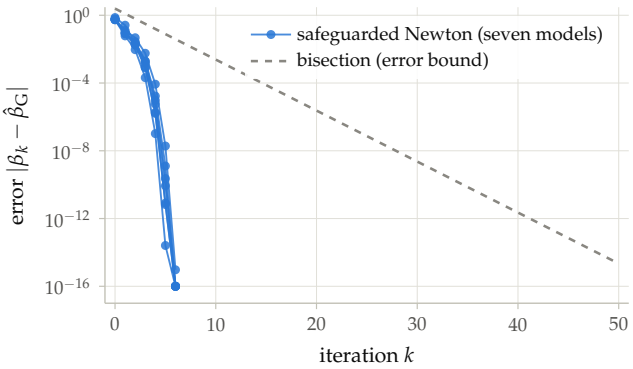


Figure 11. Convergence of Algorithm 2 for the seven models, started from $\beta_0 = 1$, and the error bound of bisection on the bracket $[0.05, 5]$. The errors are measured against the bisection solution after 50 steps; values below 10^{-16} are drawn at 10^{-16}.

Calibration lowers the ECE in all 14 combinations of model and test set: pooled, from 15.0% to 6.4% for the local probabilities and from 16.0% to 7.3% for the global marginals. For the global marginals, the 95% bootstrap intervals over the test sentences are [14.4; 17.8] before and [6.2; 8.5] after calibration, the interval of the reduction is [6.4; 10.9] points, and propagating the uncertainty of $\hat{\beta}$ through the development bootstrap gives [6.7; 8.4] for the calibrated value. Equal-mass binning gives practically the same values, 16.0% and

7.5%, and the Brier score of the decoded arcs falls from 0.217 to 0.196. The improvement is not limited to the decoded arcs: the Brier score over all candidate heads of every word, a proper scoring rule for the whole head distribution, falls from 0.503 to 0.476 for the global marginals and from 0.497 to 0.469 for the local probabilities. The negative log-likelihood of the gold trees falls from 1.30 to 0.93 nats per word on UzUDT and from 1.71 to 1.08 on UT, and on average over the models it is lower for the calibrated tree distribution than for the calibrated local model, 0.93 against 0.98 on UzUDT and 1.08 against 1.15 on UT. By Proposition 5, none of these changes alters a single decoded tree.

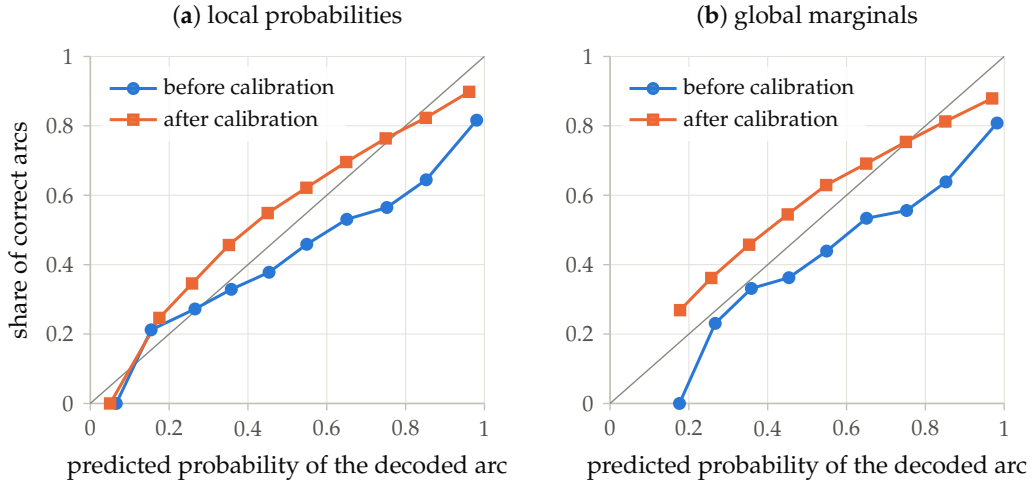


Figure 12. Reliability of the probabilities of the decoded arcs before and after temperature calibration (seven models, both test sets, 22,848 arcs): share of correct arcs in ten probability bins for (**a**) the local probabilities and (**b**) the global marginals; the diagonal corresponds to perfect calibration.

The label distributions are overconfident as well: their fitted inverse temperatures lie between 0.27 and 0.44. For a labeled arc of the decoded tree, the probability of a correct head and label is the product of the head marginal and the probability of the label; before calibration its mean is 0.755, against a share of 50.6% of correct labeled arcs, and calibrating both distributions lowers the ECE of the labeled arcs from 24.9% to 8.9% and their Brier score from 0.277 to 0.214.

Table 8 compares the calibration methods. Isotonic regression reaches the lowest pooled ECE, 3.8%, but it does not define a distribution over trees, so it gives neither tree probabilities nor a likelihood, and it transfers less well to a new genre: on UT, for the four models trained on UzUDT alone, its ECE is 10.9% on average against 9.7% for temperature scaling. The three-parameter family (34) lowers the negative log-likelihood further, to 0.89 and 1.05, and the ECE to 5.2%. Its fitted coefficients favour head-final arcs (φ_2 between -0.08 and 0.29, positive for six models) and penalize long arcs (φ_3 between -0.37 and -0.20), which moves the most probable trees towards shorter arcs and lowers UAS by about one point (70.2 against 71.2 on UzUDT and 61.9 against 62.9 on UT). The arc scores (20) already contain direction and distance terms for every pair of words, so the global terms of (34) count these preferences twice and override long arcs that the network predicts correctly. Temperature scaling of the tree distribution is therefore the method of choice when the decoded trees must stay unchanged, and the log-linear family is preferable when only probabilities are needed.

Table 8. Calibration methods fitted on the development sets and evaluated on the decoded arcs of the test sets (seven models, 22,848 arcs): expected calibration error with ten equal-width and ten equal-mass bins, Brier score, per-word negative log-likelihood of the gold trees and UAS of the most probable trees (UzUDT/UT, averaged over the models). The second column states whether the method yields a probability distribution over dependency trees.

Method	Trees	ECE	ECE, Mass	Brier	NLL	UAS
Uncalibrated marginals ($\beta = 1$)	yes	16.0	16.0	0.217	1.30/1.71	71.2/62.9
Temperature, local probabilities	no	6.4	6.4	0.192	0.98/1.15[a]	71.2/62.9
Temperature, tree distribution (Algorithm 2)	yes	7.3	7.5	0.196	0.93/1.08	71.2/62.9
Log-linear, three parameters (34)	yes	5.2	5.4	0.189	0.89/1.05	70.2/61.9
Isotonic regression of the marginals	no	3.8	4.0	0.191	—	71.2/62.9

[a] Negative log-likelihood of the gold heads under independent head choices, which also gives positive probability to structures that are not trees.

The tree entropy (25) is a useful indicator of difficult sentences: its Spearman correlation with the share of wrong heads in a sentence, averaged over the models, is 0.52 on UzUDT and 0.43 on UT, higher than that of the sentence length (0.46 and 0.34), so the entropy can be used to select sentences for manual checking.

9.4. Mixtures of the Parsers

The seven parsers can be combined without retraining. Two mixtures of the calibrated tree distributions were fixed in advance, one of all seven models and one of the three models trained on UzUDT+UT, and their MBR trees were decoded with the averaged marginals (Section 6); the label of each chosen arc is the label that most models rank first for it. Table 4 lists the results, and the paired bootstrap of Section 8 compares them with E2.2, the best single model. With predicted tags, the mixture of all seven models gives the best UzUDT scores of this study, UAS 74.2 and LAS 57.2, against 73.1 and 56.0 for E2.2 (a gain of 1.2 points with the 95% interval $[-0.7; 3.1]$); with gold tags it reaches UAS 76.4 and LAS 61.6, against 74.2 and 60.4 ($p = 0.06$). On UT, where the four models trained on UzUDT alone are much weaker, this mixture falls below E2.2. The mixture of the merged models matches E2.2 on UT with predicted tags (LAS 53.6 against 53.8) and exceeds it with gold tags, LAS 60.7 against 59.3, a gain of 1.3 points with the interval $[0.0; 2.7]$ and $p = 0.03$. Averaging tree distributions thus pays off when the components are of comparable quality on the test domain, at the cost of one matrix-tree computation per model and sentence.

9.5. Training Dynamics

Figure 13 shows the training curves of the six models with training logs. The loss per word decreases from 11.7–15.1 after 100 steps to 3.4–5.8 at the step with the best development LAS, reached after 800–1500 steps or 2.6–13.6 min of training. Afterwards the training loss keeps decreasing, down to 1.7–4.3, and the development LAS falls by 0.5–4.4 points: the network starts to fit the small training sets, and only the validation-based selection of the parameters protects the model. The switch to AMSGrad took place after 3800–4500 steps and training stopped after 6800–7500 steps, 22–104 min in total, with 0.2–0.9 s per step.

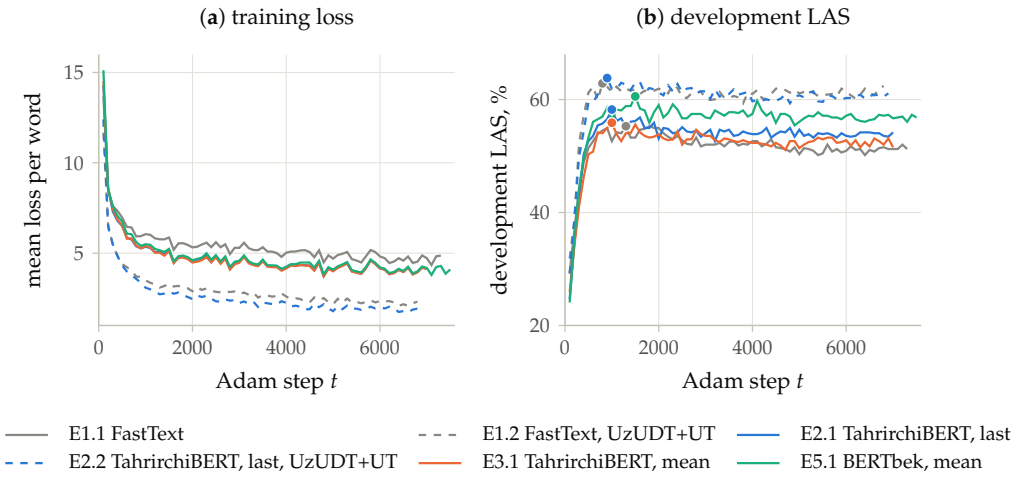


Figure 13. Training of the parsers by the stochastic method (36): (**a**) training loss per word; (**b**) LAS on the development set, with dots at the stored best parameters. Colors mark the encoder and dashed lines the models trained on UzUDT+UT.

9.6. Robustness

Five perturbation operators χ were applied to the words of the UzUDT test set: removal of the apostrophes of *o‘* and *g‘* (8.6% of the tokens changed), replacement of some function words by Russian equivalents (0.3%), random change of the case of letters (53.3%), removal of punctuation (19.5%), and lower-casing combined with the removal of punctuation (19.6%). The sensitivity of a model is $S_\chi = \mathrm{LAS}(\mathcal{D}) - \mathrm{LAS}(\chi(\mathcal{D}))$, where $\mathcal{D}$ is the test set (Table 9). The BERT-based models are insensitive to the missing apostrophes (-0.5 to $+0.2$ points), the FastText model trained on UzUDT alone loses 1.3 points, and the Russian words practically do not matter. Punctuation is an important structural signal: without it LAS falls by 4.8–10.1 points. The largest differences appear under random case changes: the models trained on UzUDT alone lose 4.7–5.6 points, BERTbek with its cased vocabulary 10.7 points, and the models trained on the merged data 16.3–17.9 points. The reason lies in the data: UzUDT is distributed in lower case and UT keeps the capitalization (96.6% of its sentences begin with a capital letter against 0.4% in UzUDT), so in the merged training data capital letters mark a genre and proper names, and random capitals shift the input towards that domain.

Table 9. Sensitivity of LAS to perturbation operators on the UzUDT test set (predicted tags, exact decoding): LAS on clean text and its change under each operator.

Model	Clean	No Diacritics	Russian Words	Random Case	No Punctuation	Lower Case, No Punct.
Tokens changed, %	—	8.6	0.3	53.3	19.5	19.6
E1.1	49.38	−1.31	+0.00	−4.72	−10.14	−10.14
E2.1	53.07	+0.00	−0.06	−5.59	−9.17	−9.17
E3.1	53.01	−0.54	−0.05	−4.94	−9.38	−9.38
E5.1	54.64	+0.00	−0.16	−10.69	−7.24	−7.24
E1.2	51.38	−0.16	−0.05	−16.27	−4.92	−4.79
E2.2	56.00	−0.33	−0.06	−17.91	−9.07	−8.93
E3.2	54.86	+0.21	−0.17	−16.99	−8.06	−8.06

10. Discussion

The exact laws of Section 4 give the structure of Uzbek syntax a precise scale. Arcs half as long as chance and a head-final order were expected for a verb-final agglutinative language. The new laws show what the tree shape and projectivity explain and what they do not. Random reorderings of the gold trees would contain 17 crossing pairs of arcs per sentence and would be projective with a probability of order 10^{-10} for sentences of 21–30 words; the gold trees contain 0.19 crossing pairs per sentence and are projective in 71% of such sentences. The covariance of Proposition 1 attributes, to first order, somewhat more than half of the missing crossings to the short arcs. Theorem 1 shows that projectivity alone places the root at an end of the sentence two times in three; Uzbek trees keep this preference for the right end only and are entirely head-final seven times more often than random projective trees. Projective decoders therefore search a tiny but linguistically relevant part of the feasible set, while the 9.2% of non-projective gold trees justify an exact non-projective decoder. The two null models are complementary: U is the reference of a decoder that searches all trees and the starting point $\beta = 0$ of the tempered family, and RLA separates the effect of word order from the effect of tree shape.

The experiments also separate the role of exactness from the role of representation. Exact decoding does not make the trees more accurate than greedy selection, but it guarantees that every output is a tree at a cost of a fraction of a millisecond, which matters for models that use the trees further, such as syntax-aware sentiment classifiers [2,3] or template-based idiom extraction [6]. Global normalization provides quantitative measures of uncertainty, the mass $Z(x)$ of well-formed trees and the tree entropy, at the price of one determinant per sentence, and Proposition 4 explains why it raises the confidence of a locally trained model. Temperature calibration removes most of this overconfidence for heads and labels with one parameter each, fitted by a convergent Newton method in a few iterations, and it leaves the decoded trees untouched, which makes the calibrated marginals usable as confidence scores for annotation and for downstream models. Richer recalibration maps can reach a lower calibration error, but either they give up the distribution over trees, as isotonic regression does, or they change the decoded trees, as the log-linear family does. Calibrated tree distributions can also be averaged, and the mixture of the seven parsers gives the best UzUDT scores of the study without any retraining. The accuracy itself is governed by the encoder and the amount of training data, and the aggregation operator brings a small gain.

The study has several limitations. The merged models were trained on gold tags, the other models on tags of a different tagger; the test sets are small, and a single LAS value has an interval of about ± 3 points; the encoders were not fine-tuned; and the network parameters were estimated with the local objective, the global distribution being fitted only through the calibration parameters. The inverse temperatures were fitted on 48 or 78 development sentences; their bootstrap intervals exclude one for every model but are about ± 0.08 wide. Finally, the effect of short arcs on crossings was estimated by a linear prediction under U, not by a null model that conditions on the arc lengths.

11. Conclusions

This paper treats Uzbek dependency parsing as a problem on spanning arborescences. The feasible analyses of a sentence of n words are the n^{n-1} single-root arborescences of a complete digraph. For their uniform distribution we derived the covariance of total arc length and crossings, and we counted projective trees by the number of head-final arcs and by the position of the root. Against these laws and random arrangements of the observed trees, the 1184 gold Uzbek trees have short arcs, are head-final and shallow, contain almost no crossings, and their root is the last content word in 73% of the sentences. Edmonds' optimum branching algorithm with root preselection decodes a sentence exactly

in less than 0.2 ms, and greedy selection leaves up to 20% of the sentences without a valid tree. The matrix-tree theorem measures the probability mass that a locally trained model wastes on ill-formed structures and yields the moments that make Newton's method applicable to calibration; the determinant and the condition number of the Laplacian matrix are controlled along the tempered path by the contraction loss of the decoder. A convex one-parameter calibration, which reduces to the uniform model at zero inverse temperature, halves the calibration error of the tree marginals without changing any tree, and a second temperature does the same for the labels. On a contamination-free test set, contextual encoders, the last-piece operator and cross-treebank training raise LAS by 3.6, 1.3 and 5.4 points, and the mixture of the calibrated parsers reaches LAS 57.2 and UAS 74.2 on UzUDT end to end, and LAS 61.6 and UAS 76.4 with gold tags. Future work will estimate all parameters with the globally normalized objective given by the matrix-tree theorem [20,62], compare monolingual with multilingual encoders, extend the exact laws to mildly non-projective classes of trees [49] and to null models that keep the arc lengths, and connect the calibrated parser with lexical and semantic models of Turkic languages.

Author Contributions: The author is solely responsible for the conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing—original draft preparation, writing—review and editing, and visualization. The author has read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Both treebanks are distributed by the Universal Dependencies project (https://universaldependencies.org, accessed on 25 October 2026). The trained models are available at https://huggingface.co/Sanatbek/uzudt and the source code, including the scripts that produce every number, table and figure of this paper, at https://github.com/SanatbekMatlatipov/robust-parsing-uzbek (both accessed on 25 October 2026).

Acknowledgments: During the preparation of this manuscript, the author used Claude (Anthropic) for the purposes of drafting and editing the text, writing analysis and plotting scripts, and checking bibliographic metadata. The author has reviewed and edited the output and takes full responsibility for the content of this publication.

Conflicts of Interest: The author declares no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AIC	Akaike information criterion
BERT	Bidirectional encoder representations from transformers
BPE	Byte-pair encoding
CLE	Chu–Liu/Edmonds optimum branching algorithm
ECE	Expected calibration error
LAS	Labeled attachment score
MBR	Minimum Bayes risk
NLL	Negative log-likelihood
OEIS	On-Line Encyclopedia of Integer Sequences
RLA	Random linear arrangement
UAS	Unlabeled attachment score
UD	Universal Dependencies

References

1. de Marneffe, M.C.; Manning, C.D.; Nivre, J.; Zeman, D. Universal Dependencies. *Comput. Linguist.* **2021**, *47*, 255–308. https://doi.org/10.1162/coli_a_00402.
2. Chen, J.; Qiu, Z.; Liu, J.; Xue, Y.; Cai, Q. Syntactic Structure-Enhanced Dual Graph Convolutional Network for Aspect-Level Sentiment Classification. *Mathematics* **2023**, *11*, 3877. https://doi.org/10.3390/math11183877.
3. Yang, J.; Dai, A.; Xue, Y.; Zeng, B.; Liu, X. Syntactically Enhanced Dependency-POS Weighted Graph Convolutional Network for Aspect-Based Sentiment Analysis. *Mathematics* **2022**, *10*, 3353. https://doi.org/10.3390/math10183353.
4. Qing-dao-er-ji, R.; Cheng, K.; Pang, R. Research on Traditional Mongolian-Chinese Neural Machine Translation Based on Dependency Syntactic Information and Transformer Model. *Appl. Sci.* **2022**, *12*, 10074. https://doi.org/10.3390/app121910074.
5. Donaj, G.; Sepesy Maučec, M. On the Use of Morpho-Syntactic Description Tags in Neural Machine Translation with Small and Large Training Corpora. *Mathematics* **2022**, *10*, 1608. https://doi.org/10.3390/math10091608.
6. Nazirova, E.Sh.; Madetbayeva, N.B.; Mirzayeva, F.Sh.; Islomova, M.B. A Mathematical Model for the Automatic Extraction of Idioms in Turkic Languages. *Probl. Comput. Appl. Math.* **2026**, *74*, 152–170. https://doi.org/10.71310/pcam.4_74.2026.10.
7. Akhmedjanova, D.A. A Semantic Linking Model for Turkic Languages: A Fairness-Constrained Cross-Lingual Approach for Low-Resource Settings. *Probl. Comput. Appl. Math.* **2026**, *74*, 171–181. https://doi.org/10.71310/pcam.4_74.2026.11.
8. McDonald, R.; Pereira, F.; Ribarov, K.; Hajič, J. Non-Projective Dependency Parsing Using Spanning Tree Algorithms. In Proceedings of the Human Language Technology Conference and Conference on Empirical Methods in Natural Language Processing, Vancouver, BC, Canada, 2005; pp. 523–530. Available online: https://aclanthology.org/H05-1066/ (accessed on 24 October 2026).
9. Wang, Z.; Ouyang, D.; Wang, Y.; Liang, Q.; Huang, Z. Efficient and Effective Directed Minimum Spanning Tree Queries. *Mathematics* **2023**, *11*, 2200. https://doi.org/10.3390/math11092200.
10. Abdelmaguid, T.F. An Efficient Mixed Integer Linear Programming Model for the Minimum Spanning Tree Problem. *Mathematics* **2018**, *6*, 183. https://doi.org/10.3390/math6100183.
11. Luo, M.; Qin, H.; Wu, X.; Xiong, C.; Xia, D.; Ke, Y. Efficient Maintenance of Minimum Spanning Trees in Dynamic Weighted Undirected Graphs. *Mathematics* **2024**, *12*, 1021. https://doi.org/10.3390/math12071021.
12. Hosseini, A. Uncertainty Modeling and Stability Assessment of Minimum Spanning Trees in Network Design. *Mathematics* **2024**, *12*, 3812. https://doi.org/10.3390/math12233812.
13. Chu, Y.J.; Liu, T.H. On the Shortest Arborescence of a Directed Graph. *Sci. Sin.* **1965**, *14*, 1396–1400.
14. Edmonds, J. Optimum Branchings. *J. Res. Natl. Bur. Stand. Sect. B Math. Math. Phys.* **1967**, *71B*, 233–240. https://doi.org/10.6028/jres.071B.032.
15. Tarjan, R.E. Finding Optimum Branchings. *Networks* **1977**, *7*, 25–35. https://doi.org/10.1002/net.3230070103.
16. Zmigrod, R.; Vieira, T.; Cotterell, R. Please Mind the Root: Decoding Arborescences for Dependency Parsing. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Online, 2020; pp. 4809–4819. https://doi.org/10.18653/v1/2020.emnlp-main.390.
17. Stanojević, M.; Cohen, S.B. A Root of a Problem: Optimizing Single-Root Dependency Parsing. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Online and Punta Cana, Dominican Republic, 2021; pp. 10540–10557. https://doi.org/10.18653/v1/2021.emnlp-main.823.
18. Kirchhoff, G. Ueber die Auflösung der Gleichungen, auf welche man bei der Untersuchung der linearen Vertheilung galvanischer Ströme geführt wird. *Ann. Phys.* **1847**, *148*, 497–508. https://doi.org/10.1002/andp.18471481202.
19. Tutte, W.T. The Dissection of Equilateral Triangles into Equilateral Triangles. *Math. Proc. Camb. Philos. Soc.* **1948**, *44*, 463–482. https://doi.org/10.1017/S030500410002449X.
20. Koo, T.; Globerson, A.; Carreras, X.; Collins, M. Structured Prediction Models via the Matrix-Tree Theorem. In Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL), Prague, Czech Republic, 2007; pp. 141–150. Available online: https://aclanthology.org/D07-1015/ (accessed on 24 October 2026).
21. Smith, D.A.; Smith, N.A. Probabilistic Models of Nonprojective Dependency Trees. In Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL), Prague, Czech Republic, 2007; pp. 132–140. Available online: https://aclanthology.org/D07-1014/ (accessed on 24 October 2026).
22. McDonald, R.; Satta, G. On the Complexity of Non-Projective Data-Driven Dependency Parsing. In Proceedings of the Tenth International Conference on Parsing Technologies (IWPT), Prague, Czech Republic, 2007; pp. 121–132. https://doi.org/10.3115/1621410.1621426.
23. Zmigrod, R.; Vieira, T.; Cotterell, R. Efficient Computation of Expectations under Spanning Tree Distributions. *Trans. Assoc. Comput. Linguist.* **2021**, *9*, 675–690. https://doi.org/10.1162/tacl_a_00391.

24. Dozat, T.; Manning, C.D. Deep Biaffine Attention for Neural Dependency Parsing. In Proceedings of the 5th International Conference on Learning Representations (ICLR), Toulon, France, 2017. Available online: https://arxiv.org/abs/1611.01734 (accessed on 25 October 2026).
25. Qi, P.; Zhang, Y.; Zhang, Y.; Bolton, J.; Manning, C.D. Stanza: A Python Natural Language Processing Toolkit for Many Human Languages. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, Online, 2020; pp. 101–108. https://doi.org/10.18653/v1/2020.acl-demos.14.
26. Akhundjanova, A.; Talamo, L. Universal Dependencies Treebank for Uzbek. In Proceedings of the 3rd Workshop on Resources and Representations for Under-Resourced Languages and Domains (RESOURCEFUL-2025), Tallinn, Estonia, 2025; pp. 1–6. Available online: https://aclanthology.org/2025.resourceful-1.1/ (accessed on 24 October 2026).
27. Matlatipov, S.; Aripov, M.; Bobokandov, M.; Matlatipov, G. Annotated Universal Dependencies Dataset for Literary and Educational Uzbek Texts. *Data Brief* **2026**, *66*, 112857. https://doi.org/10.1016/j.dib.2026.112857.
28. Matlatipov, S.; Aripov, M. UzUDT: Uzbek Universal Dependencies Treebank. In Proceedings of the 15th Language Resources and Evaluation Conference (LREC 2026), Palma de Mallorca, Spain, 2026; pp. 11642–11649. https://doi.org/10.63317/2uedjqezjxn5.
29. Matlatipov, S.; Kurbanova, M. A Gold-Standard Dependency Treebank for the Uzbek Language. In Proceedings of the 2026 IEEE Ural-Siberian Conference on Biomedical Engineering, Radioelectronics and Information Technology (USBEREIT), Yekaterinburg, Russia, 2026; pp. 1–3. https://doi.org/10.1109/USBEREIT70063.2026.11580531.
30. Mamasaidov, M.; Shopulatov, A. TahrirchiBERT Base: A Pretrained Language Model for Uzbek. Hugging Face model repository, 2023. Available online: https://huggingface.co/tahrirchi/tahrirchi-bert-base (accessed on 25 October 2026).
31. Kuriyozov, E.; Vilares, D.; Gómez-Rodríguez, C. BERTbek: A Pretrained Language Model for Uzbek. In Proceedings of the 3rd Annual Meeting of the Special Interest Group on Under-Resourced Languages @ LREC-COLING 2024, Torino, Italy, 2024; pp. 33–44. Available online: https://aclanthology.org/2024.sigul-1.5/ (accessed on 24 October 2026).
32. Salaev, U. UzMorphAnalyser: A Morphological Analysis Model for the Uzbek Language Using Inflectional Endings. *AIP Conf. Proc.* **2024**, *3244*, 030058. https://doi.org/10.1063/5.0241461.
33. Mengliev, D.; Barakhnin, V.; Abdurakhmonova, N. Development of Intellectual Web System for Morph Analyzing of Uzbek Words. *Appl. Sci.* **2021**, *11*, 9117. https://doi.org/10.3390/app11199117.
34. Madatov, K.; Sattarova, S.; Vičič, J. TF-IDF-Based Classification of Uzbek Educational Texts. *Appl. Sci.* **2025**, *15*, 10808. https://doi.org/10.3390/app151910808.
35. Saidov, B.; Barakhnin, V.; Saparbaev, R.; Narmuratov, Z.; Manzura, R.; Zilolakhon, R.; Atajanova, A. A Hybrid NER–Sentiment Model for Uzbek Texts: Integrating Lexical, Deep Learning, and Entity-Based Approaches. *Big Data Cogn. Comput.* **2026**, *10*, 92. https://doi.org/10.3390/bdcc10030092.
36. Matlatipov, S. Towards Robust Uzbek Neural Dependency Parsing: Cross-Treebank Training. In *Natural Language Processing and Information Systems*; Cabrio, E.; Monteiro, E., Eds.; Springer: Cham, Switzerland, 2026; pp. 204–218. https://doi.org/10.1007/978-3-032-29532-3_15.
37. Matlatipov, S.; Ismoilova, M.; Aripov, M. Graph-Based vs Transition-Based Dependency Parsing for Uzbek. In Proceedings of the 2026 IEEE 27th International Conference of Young Professionals in Electron Devices and Materials (EDM), Altai, Russia, 2026; pp. 1–6. https://doi.org/10.1109/EDM69524.2026.11632281.
38. Matlatipov, S. Neural Morphosyntactic Tagging and Dependency Parsing for Uzbek. In Proceedings of the 2026 IEEE Ural-Siberian Conference on Biomedical Engineering, Radioelectronics and Information Technology (USBEREIT), Yekaterinburg, Russia, 2026; pp. 1–4. https://doi.org/10.1109/USBEREIT70063.2026.11580890.
39. Ferrer-i Cancho, R. Euclidean Distance between Syntactically Linked Words. *Phys. Rev. E* **2004**, *70*, 056135. https://doi.org/10.1103/PhysRevE.70.056135.
40. Ferrer-i Cancho, R.; Gómez-Rodríguez, C.; Esteban, J.L. Are Crossing Dependencies Really Scarce? *Phys. A* **2018**, *493*, 311–329. https://doi.org/10.1016/j.physa.2017.10.048.
41. Ferrer-i Cancho, R.; Gómez-Rodríguez, C.; Esteban, J.L.; Alemany-Puig, L. Optimality of Syntactic Dependency Distances. *Phys. Rev. E* **2022**, *105*, 014308. https://doi.org/10.1103/PhysRevE.105.014308.
42. Dell'Amico, M.; Jamal, J.; Montemanni, R. Compact Models to Solve the Precedence-Constrained Minimum-Cost Arborescence Problem with Waiting Times. *Algorithms* **2024**, *17*, 12. https://doi.org/10.3390/a17010012.
43. Prüfer, H. Neuer Beweis eines Satzes über Permutationen. *Arch. Math. Phys.* **1918**, *27*, 742–744.
44. Moon, J.W. *Counting Labelled Trees*; Vol. 1, *Canadian Mathematical Monographs*, Canadian Mathematical Congress: Montreal, QC, Canada, 1970.
45. Pitman, J. Coalescent Random Forests. *J. Comb. Theory Ser. A* **1999**, *85*, 165–193. https://doi.org/10.1006/jcta.1998.2919.
46. Flajolet, P.; Sedgewick, R. *Analytic Combinatorics*; Cambridge University Press: Cambridge, UK, 2009. https://doi.org/10.1017/CBO9780511801655.
47. Flajolet, P.; Noy, M. Analytic Combinatorics of Non-Crossing Configurations. *Discrete Math.* **1999**, *204*, 203–229. https://doi.org/10.1016/S0012-365X(98)00372-0.

48. OEIS Foundation Inc. Sequence A006013, a(n) = binomial(3n+1, n)/(n+1), The On-Line Encyclopedia of Integer Sequences. Available online: https://oeis.org/A006013 (accessed on 25 October 2026).
49. Kuhlmann, M. Mildly Non-Projective Dependency Grammar. *Comput. Linguist.* **2013**, *39*, 355–387. https://doi.org/10.1162/COLI_a_00125.
50. Qi, F.; Cerone, P. Some Properties of the Fuss–Catalan Numbers. *Mathematics* **2018**, *6*, 277. https://doi.org/10.3390/math6120277.
51. Selim, A.; Saracevic, M.; Stosic, L.; Aydin, O.; Zajmović, M. Bidirectional Algorithms for Polygon Triangulations and (m + 2)-Angulations via Fuss–Catalan Numbers. *Mathematics* **2025**, *13*, 3837. https://doi.org/10.3390/math13233837.
52. Selim, A.; Saracevic, M.; Susuri, A. Limiting the Number of Possible CFG Derivative Trees During Grammar Induction with Catalan Numbers. *Mathematics* **2026**, *14*, 249. https://doi.org/10.3390/math14020249.
53. Rényi, A.; Szekeres, G. On the Height of Trees. *J. Aust. Math. Soc.* **1967**, *7*, 497–507. https://doi.org/10.1017/S1446788700004432.
54. Huillet, T.E. Statistical Aspects of Two Classes of Random Binomial Trees and Forests. *Mathematics* **2025**, *13*, 291. https://doi.org/10.3390/math13020291.
55. Ferrer-i Cancho, R. The Sum of Edge Lengths in Random Linear Arrangements. *J. Stat. Mech.* **2019**, *2019*, 053401. https://doi.org/10.1088/1742-5468/ab11e2.
56. Gómez-Rodríguez, C.; Ferrer-i Cancho, R. Scarcity of Crossing Dependencies: A Direct Outcome of a Specific Constraint? *Phys. Rev. E* **2017**, *96*, 062304. https://doi.org/10.1103/PhysRevE.96.062304.
57. Alemany-Puig, L.; Ferrer-i Cancho, R. Linear-Time Calculation of the Expected Sum of Edge Lengths in Random Projective Linearizations of Trees. *Comput. Linguist.* **2022**, *48*, 491–516. https://doi.org/10.1162/coli_a_00442.
58. Li, W.; Liu, H. Interactions Among Morphology, Word Order, and Syntactic Directionality: Evidence from 55 Languages. *Entropy* **2025**, *27*, 1128. https://doi.org/10.3390/e27111128.
59. Wu, F. Dependency Parsing with Transformed Feature. *Information* **2017**, *8*, 13. https://doi.org/10.3390/info8010013.
60. Li, B.; Fan, Y.; Sataer, Y.; Gao, Z.; Gui, Y. Improving Semantic Dependency Parsing with Higher-Order Information Encoded by Graph Neural Networks. *Appl. Sci.* **2022**, *12*, 4089. https://doi.org/10.3390/app12084089.
61. Li, R.; Shu, S.; Wang, S.; Liu, Y.; Li, Y.; Peng, M. DAT-MT Accelerated Graph Fusion Dependency Parsing Model for Small Samples in Professional Fields. *Entropy* **2023**, *25*, 1444. https://doi.org/10.3390/e25101444.
62. Zhang, Y.; Li, Z.; Zhang, M. Efficient Second-Order TreeCRF for Neural Dependency Parsing. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 2020; pp. 3295–3305. https://doi.org/10.18653/v1/2020.acl-main.302.
63. Eisner, J.M. Three New Probabilistic Models for Dependency Parsing: An Exploration. In Proceedings of the 16th International Conference on Computational Linguistics (COLING), Copenhagen, Denmark, 1996; pp. 340–345. Available online: https://aclanthology.org/C96-1058/ (accessed on 24 October 2026).
64. Yang, S.; Tu, K. Headed-Span-Based Projective Dependency Parsing. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Dublin, Ireland, 2022; pp. 2188–2200. https://doi.org/10.18653/v1/2022.acl-long.155.
65. Amini, A.; Liu, T.; Cotterell, R. Hexatagging: Projective Dependency Parsing as Tagging. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), Toronto, ON, Canada, 2023; pp. 1453–1464. https://doi.org/10.18653/v1/2023.acl-short.124.
66. Devlin, J.; Chang, M.W.; Lee, K.; Toutanova, K. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Minneapolis, MN, USA, 2019; pp. 4171–4186. https://doi.org/10.18653/v1/N19-1423.
67. Al-Ghamdi, S.; Al-Khalifa, H.; Al-Salman, A. Fine-Tuning BERT-Based Pre-Trained Models for Arabic Dependency Parsing. *Appl. Sci.* **2023**, *13*, 4225. https://doi.org/10.3390/app13074225.
68. Türk, U.; Atmaca, F.; Özateş, Ş.B.; Berk, G.; Bedir, S.T.; Köksal, A.; Öztürk Başaran, B.; Güngör, T.; Özgür, A. Resources for Turkish Dependency Parsing: Introducing the BOUN Treebank and the BoAT Annotation Tool. *Lang. Resour. Eval.* **2022**, *56*, 259–307. https://doi.org/10.1007/s10579-021-09558-0.
69. Martín Arista, J. Toward a Universal Dependencies Treebank of Old English: Representing the Morphological Relatedness of Un-Derivatives. *Languages* **2024**, *9*, 76. https://doi.org/10.3390/languages9030076.
70. Rust, P.; Pfeiffer, J.; Vulić, I.; Ruder, S.; Gurevych, I. How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), Online, 2021; pp. 3118–3135. https://doi.org/10.18653/v1/2021.acl-long.243.
71. Tukeyev, U.; Rysbek, B. Morphology-Aware Segmentation and Tokenization for Turkic Languages: A CSE-Guided Framework (The Kazakh Case). *Information* **2026**, *17*, 128. https://doi.org/10.3390/info17020128.

72. Ibiyev, K. Cross-Family Stem-Suffix Boundary Segmentation in Turkic and Uralic Languages: A ByT5 Study of Azerbaijani, Turkish, Hungarian, and Finnish. *Information* **2026**, *17*, 745. https://doi.org/10.3390/info17080745.
73. Mussakhojayeva, S.; Dauletbek, K.; Yeshpanov, R.; Varol, H.A. Multilingual Speech Recognition for Turkic Languages. *Information* **2023**, *14*, 74. https://doi.org/10.3390/info14020074.
74. Kadyrbek, N.; Tuimebayev, Z.; Mansurova, M.; Viegas, V. The Development of Small-Scale Language Models for Low-Resource Languages, with a Focus on Kazakh and Direct Preference Optimization. *Big Data Cogn. Comput.* **2025**, *9*, 137. https://doi.org/10.3390/bdcc9050137.
75. Maimaiti, M.; Wumaier, A.; Abiderexiti, K.; Yibulayin, T. Bidirectional Long Short-Term Memory Network with a Conditional Random Field Layer for Uyghur Part-Of-Speech Tagging. *Information* **2017**, *8*, 157. https://doi.org/10.3390/info8040157.
76. Sel, I.; Hanbay, D. Efficient Adaptation: Enhancing Multilingual Models for Low-Resource Language Translation. *Mathematics* **2024**, *12*, 3149. https://doi.org/10.3390/math12193149.
77. Guo, C.; Pleiss, G.; Sun, Y.; Weinberger, K.Q. On Calibration of Modern Neural Networks. In Proceedings of the 34th International Conference on Machine Learning (ICML); Precup, D.; Teh, Y.W., Eds., Sydney, Australia, 2017; Vol. 70, *Proceedings of Machine Learning Research*, pp. 1321–1330. Available online: https://proceedings.mlr.press/v70/guo17a.html (accessed on 24 October 2026).
78. Cao, Z.; Li, Y.; Kim, D.H.; Shin, B.S. Deep Neural Network Confidence Calibration from Stochastic Weight Averaging. *Electronics* **2024**, *13*, 503. https://doi.org/10.3390/electronics13030503.
79. Zadrozny, B.; Elkan, C. Transforming Classifier Scores into Accurate Multiclass Probability Estimates. In Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Edmonton, AB, Canada, 2002; pp. 694–699. https://doi.org/10.1145/775047.775151.
80. Brier, G.W. Verification of Forecasts Expressed in Terms of Probability. *Mon. Weather Rev.* **1950**, *78*, 1–3. https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.
81. Kompa, B.; Snoek, J.; Beam, A.L. Empirical Frequentist Coverage of Deep Learning Uncertainty Quantification Procedures. *Entropy* **2021**, *23*, 1608. https://doi.org/10.3390/e23121608.
82. Nguyen, K.; O'Connor, B. Posterior Calibration and Exploratory Analysis for Natural Language Processing Models. In Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing, Lisbon, Portugal, 2015; pp. 1587–1598. https://doi.org/10.18653/v1/D15-1182.
83. Kuleshov, V.; Liang, P. Calibrated Structured Prediction. In Proceedings of the Advances in Neural Information Processing Systems 28 (NIPS 2015), Montreal, QC, Canada, 2015. Available online: https://papers.nips.cc/paper/5658-calibrated-structured-prediction (accessed on 25 October 2026).
84. Jagannatha, A.; Yu, H. Calibrating Structured Output Predictors for Natural Language Processing. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 2020; pp. 2078–2092. https://doi.org/10.18653/v1/2020.acl-main.188.
85. Burr, T.; Skurikhin, A. Conditional Random Fields for Pattern Recognition Applied to Structured Data. *Algorithms* **2015**, *8*, 466–483. https://doi.org/10.3390/a8030466.
86. Smith, D.A.; Eisner, J. Minimum Risk Annealing for Training Log-Linear Models. In Proceedings of the COLING/ACL 2006 Main Conference Poster Sessions, Sydney, Australia, 2006; pp. 787–794. Available online: https://aclanthology.org/P06-2101/ (accessed on 25 October 2026).
87. Wainwright, M.J.; Jordan, M.I. Graphical Models, Exponential Families, and Variational Inference. *Found. Trends Mach. Learn.* **2008**, *1*, 1–305. https://doi.org/10.1561/2200000001.
88. Jin, L.; Ding, Z.; Zhou, H. Evaluation of Chinese Natural Language Processing System Based on Metamorphic Testing. *Mathematics* **2022**, *10*, 1276. https://doi.org/10.3390/math10081276.
89. Berg-Kirkpatrick, T.; Burkett, D.; Klein, D. An Empirical Investigation of Statistical Significance in NLP. In Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning, Jeju Island, Republic of Korea, 2012; pp. 995–1005. Available online: https://aclanthology.org/D12-1091/ (accessed on 24 October 2026).
90. Holm, S. A Simple Sequentially Rejective Multiple Test Procedure. *Scand. J. Stat.* **1979**, *6*, 65–70.
91. Cayley, A. A Theorem on Trees. *Q. J. Pure Appl. Math.* **1889**, *23*, 376–378.
92. Graham, R.L.; Knuth, D.E.; Patashnik, O. *Concrete Mathematics: A Foundation for Computer Science*, 2nd ed.; Addison-Wesley: Reading, MA, USA, 1994.
93. OEIS Foundation Inc. Sequence A001764, a(n) = binomial(3n, n)/(2n + 1), The On-Line Encyclopedia of Integer Sequences. Available online: https://oeis.org/A001764 (accessed on 25 October 2026).
94. OEIS Foundation Inc. Sequence A371395, Triangle read by rows: T(n, k) = binomial(n + k, k) binomial(2n - k, n - k)/(n + 1), The On-Line Encyclopedia of Integer Sequences. Available online: https://oeis.org/A371395 (accessed on 25 October 2026).

95. Clauset, A.; Shalizi, C.R.; Newman, M.E.J. Power-Law Distributions in Empirical Data. *SIAM Rev.* **2009**, *51*, 661–703. https://doi.org/10.1137/070710111.
96. Varin, C.; Reid, N.; Firth, D. An Overview of Composite Likelihood Methods. *Stat. Sin.* **2011**, *21*, 5–42.
97. Qi, P.; Dozat, T.; Zhang, Y.; Manning, C.D. Universal Dependency Parsing from Scratch. In Proceedings of the CoNLL 2018 Shared Task: Multilingual Parsing from Raw Text to Universal Dependencies, Brussels, Belgium, 2018; pp. 160–170. https://doi.org/10.18653/v1/K18-2016.
98. Conneau, A.; Khandelwal, K.; Goyal, N.; Chaudhary, V.; Wenzek, G.; Guzmán, F.; Grave, E.; Ott, M.; Zettlemoyer, L.; Stoyanov, V. Unsupervised Cross-Lingual Representation Learning at Scale. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 2020; pp. 8440–8451. https://doi.org/10.18653/v1/2020.acl-main.747.
99. Higham, N.J. *Accuracy and Stability of Numerical Algorithms*, 2nd ed.; Society for Industrial and Applied Mathematics: Philadelphia, PA, USA, 2002. https://doi.org/10.1137/1.9780898718027.
100. Nocedal, J.; Wright, S.J. *Numerical Optimization*, 2nd ed.; Springer: New York, NY, USA, 2006. https://doi.org/10.1007/978-0-387-40065-5.
101. Barndorff-Nielsen, O. *Information and Exponential Families in Statistical Theory*; John Wiley & Sons: Chichester, UK, 1978.
102. Kingma, D.P.; Ba, J. Adam: A Method for Stochastic Optimization. In Proceedings of the 3rd International Conference on Learning Representations (ICLR), San Diego, CA, USA, 2015. Available online: https://arxiv.org/abs/1412.6980 (accessed on 25 October 2026).
103. Reddi, S.J.; Kale, S.; Kumar, S. On the Convergence of Adam and Beyond. In Proceedings of the 6th International Conference on Learning Representations (ICLR), Vancouver, BC, Canada, 2018. Available online: https://openreview.net/forum?id=ryQu7f-RZ (accessed on 25 October 2026).
104. Zeman, D.; Hajič, J.; Popel, M.; Potthast, M.; Straka, M.; Ginter, F.; Nivre, J.; Petrov, S. CoNLL 2018 Shared Task: Multilingual Parsing from Raw Text to Universal Dependencies. In Proceedings of the CoNLL 2018 Shared Task: Multilingual Parsing from Raw Text to Universal Dependencies, Brussels, Belgium, 2018; pp. 1–21. https://doi.org/10.18653/v1/K18-2001.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.